

CSI 2110
Data Structures and Algorithms

Lecture 4: Trees and Binary Trees

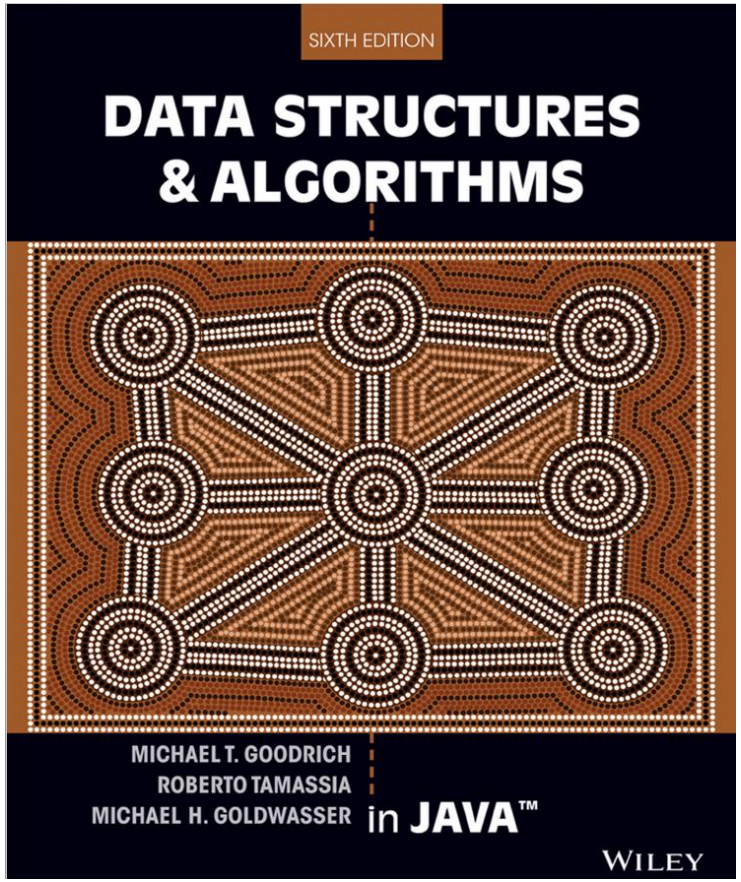
Bo Sun

Fall 2026

A note on the slides

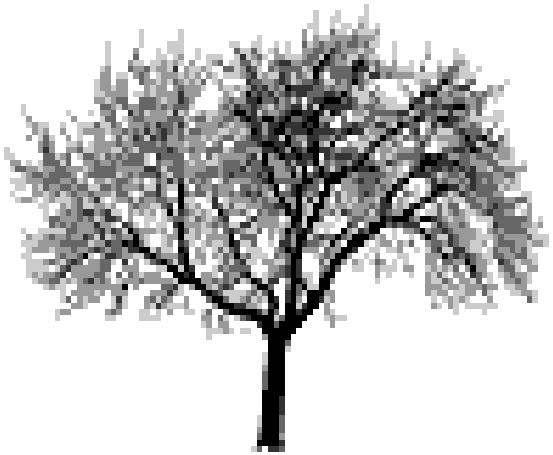
Adapted from the slides that accompany this book.

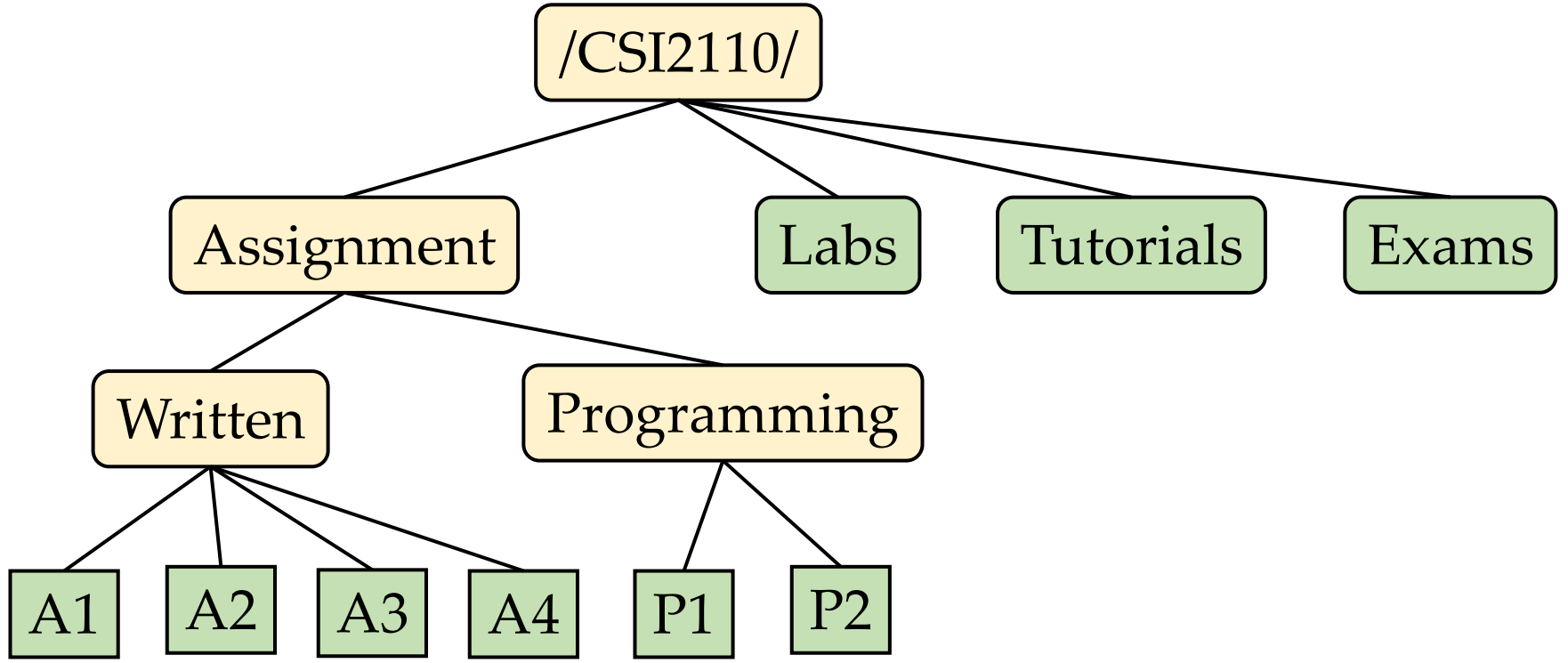
Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.

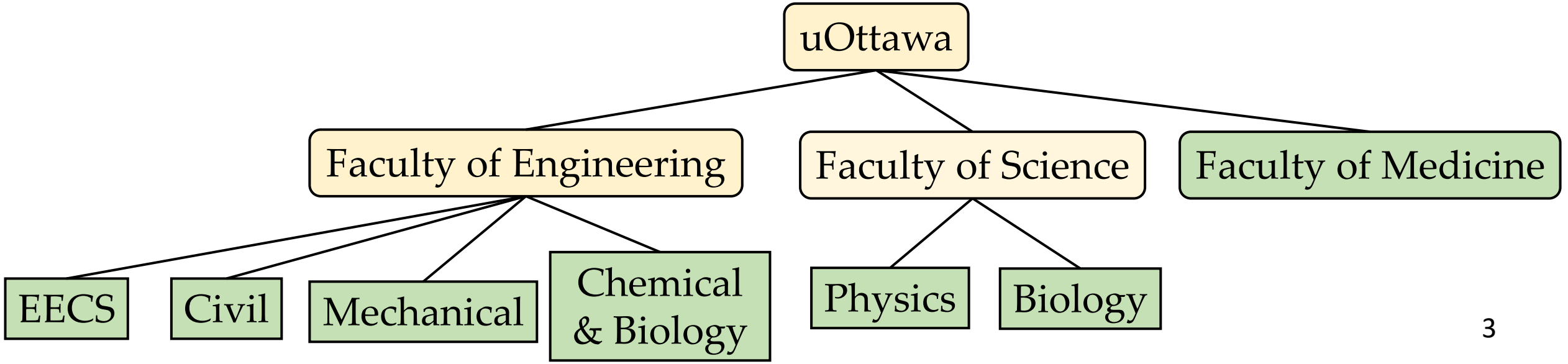


Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

Trees




/CSI2110/
Assignment
Labs
Tutorials
Exams
Written
Programming
A1
A2
A3
A4
P1
P2

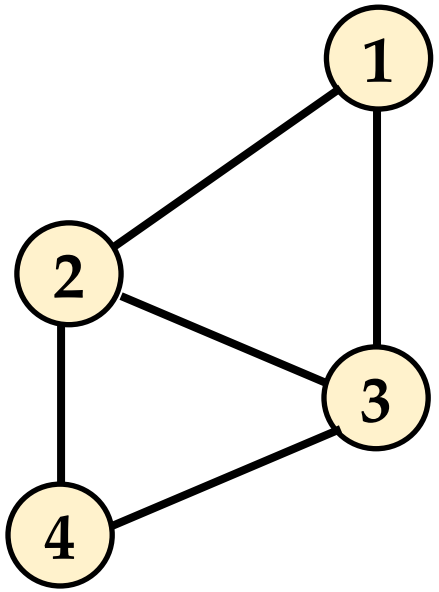

uOttawa
Faculty of Engineering
Faculty of Science
Faculty of Medicine
EECS
Civil
Mechanical
Chemical & Biology
Physics
Biology

Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
- Traversals of Trees

Graphs: Nodes and Edges

- A graph is a pair (V, E), where
  - V is a set of vertices/nodes
  - E is a set of edges/links, each edge is a pair of vertices (u, v), where $u, v \in V$

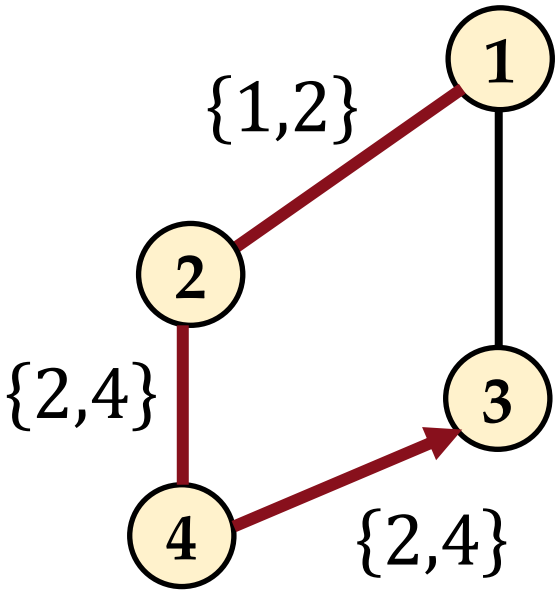


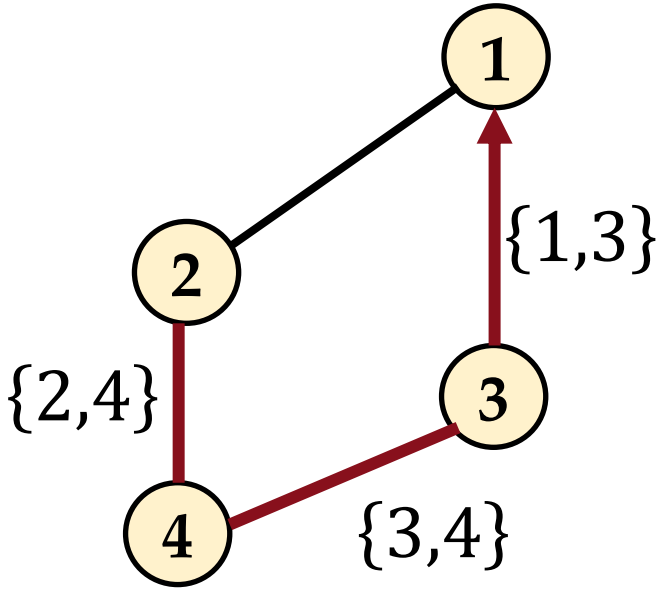
$$V = \{1, 2, 3, 4\}$$

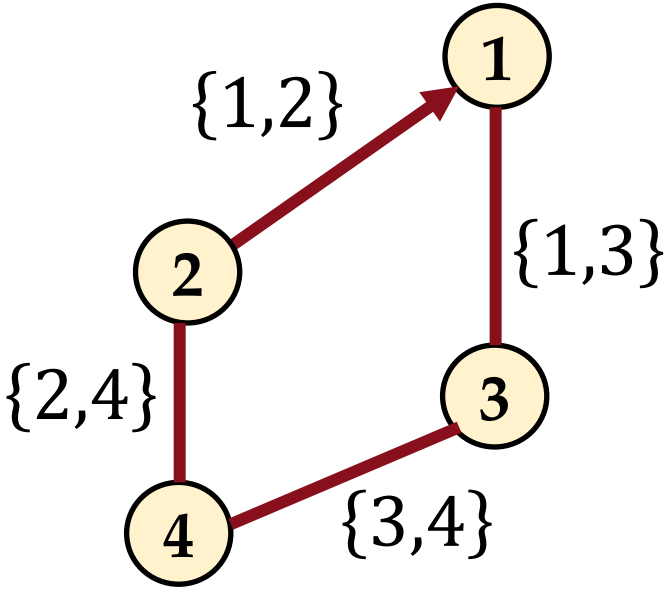
$$E = \{\{1,2\},\ \{2,3\},\ \{2,4\},\ \{3,4\}, \{1,3\}\}$$

Graphs: Path and Cycle

- A path: a sequence of nodes $v_0, v_1, \ldots, v_k$ such that $\{v_i, v_{i+1}\} \in E$, for all $i = 0,1, \ldots, k-1$.
- A cycle: a path where $v_0 = v_k$



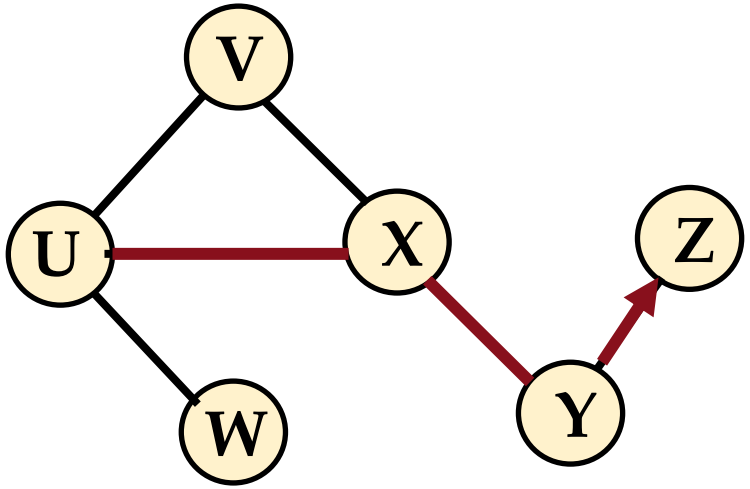




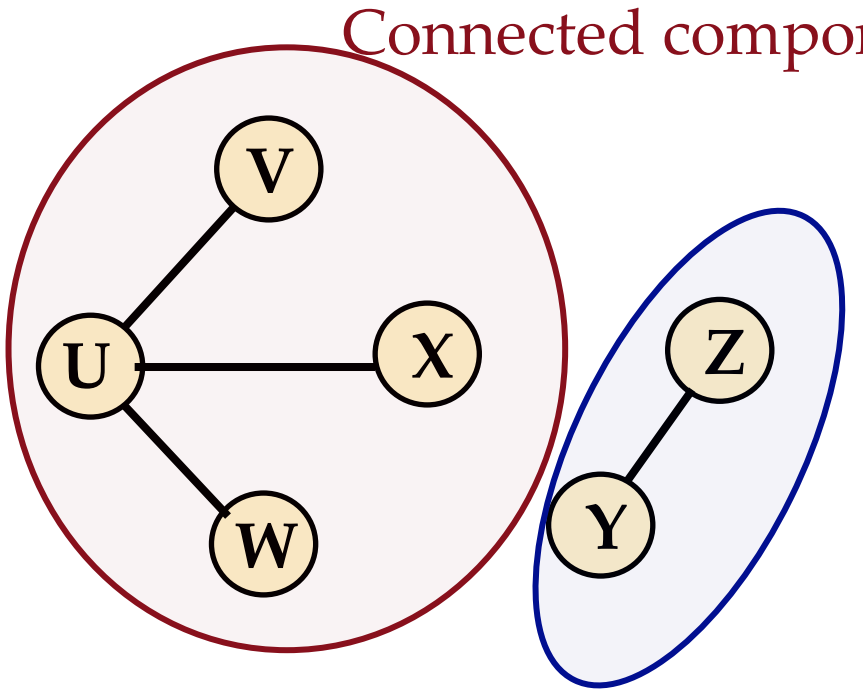
Path $P = (1, 2, 4, 3)$ Path $P = (2, 4, 3, 1)$ Cycle $C = (1, 3, 4, 2, 1)$

Connectivity in Graphs

- Connected: vertex u is connected to v if there exists a path from u to v
- G is a connected graph if for every $u, v \in V$, u is connected to v



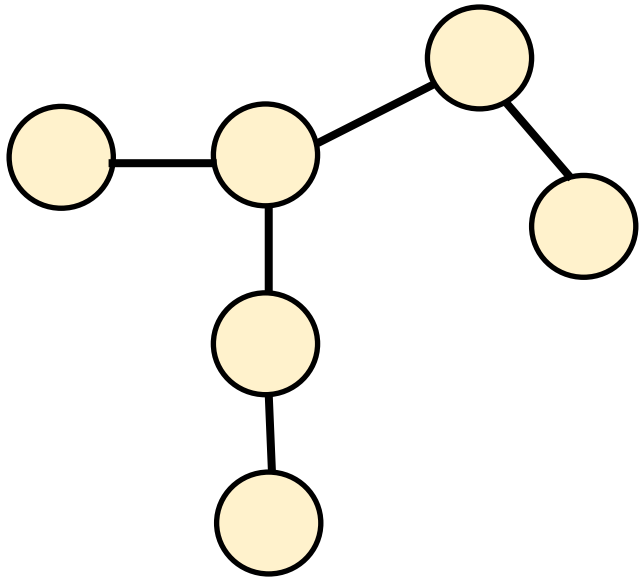
Connected graph



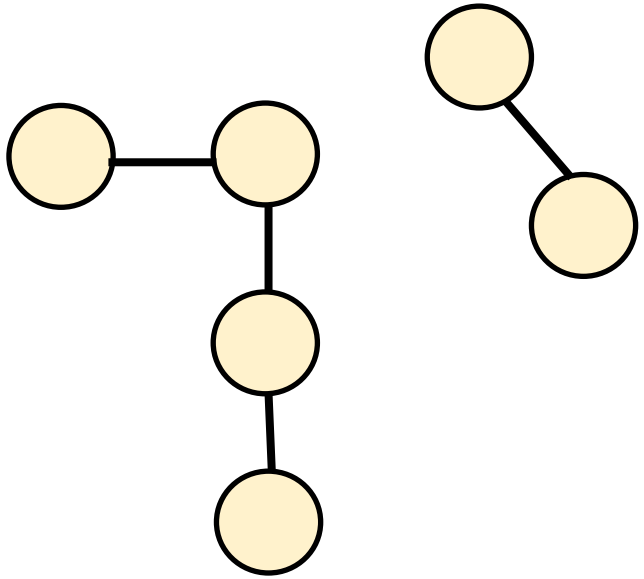
Unconnected graph

Tree

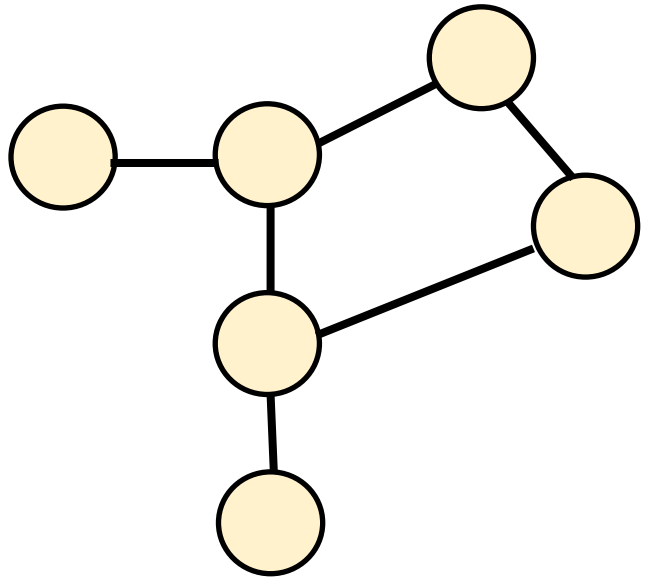
- A tree is a connected graph with no cycles



Tree



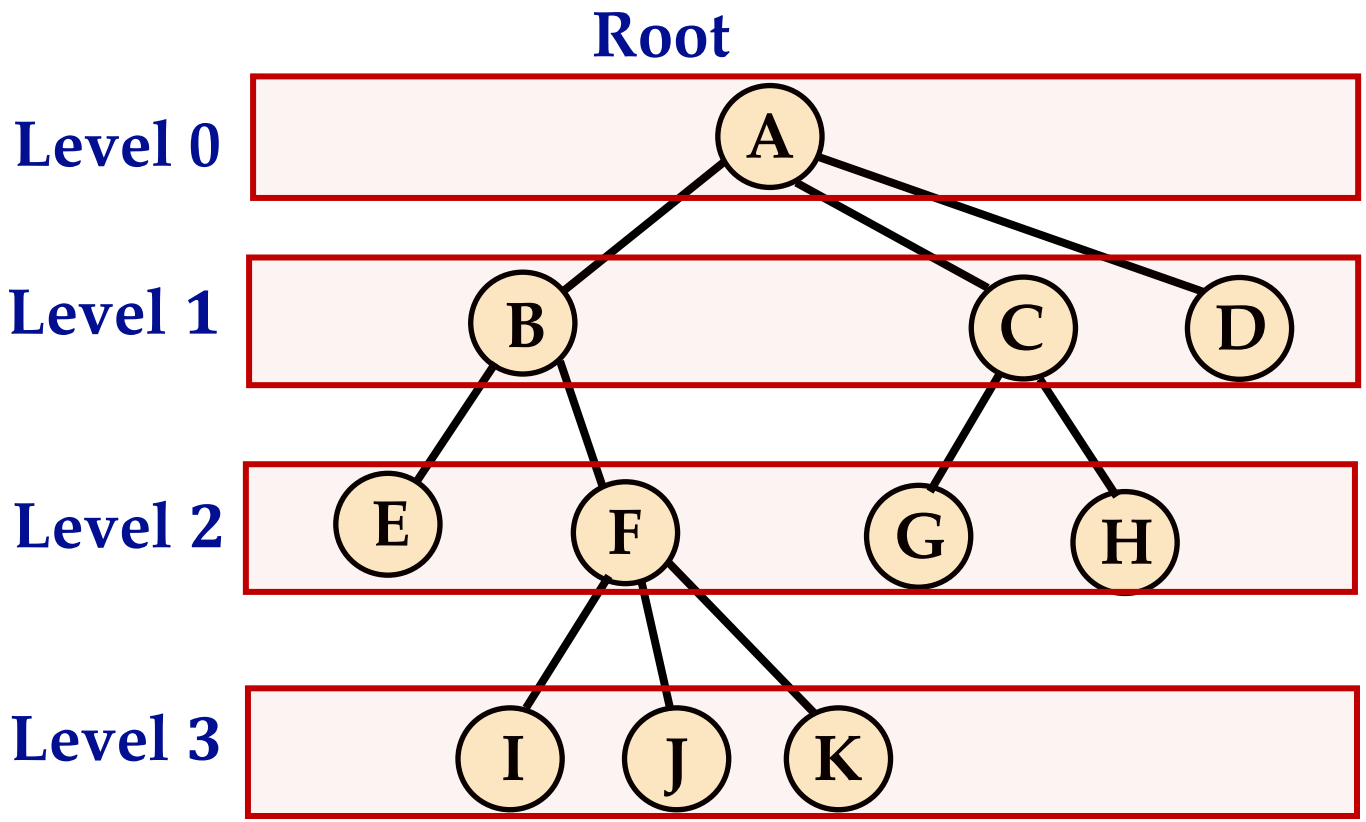
Not a tree
(not connected)



Not a tree
(with cycles)

(Rooted) Tree

- Distance between two nodes: number of edges
- Depth of a node: distance from the root
- Height of a tree: maximum depth among all nodes in the tree

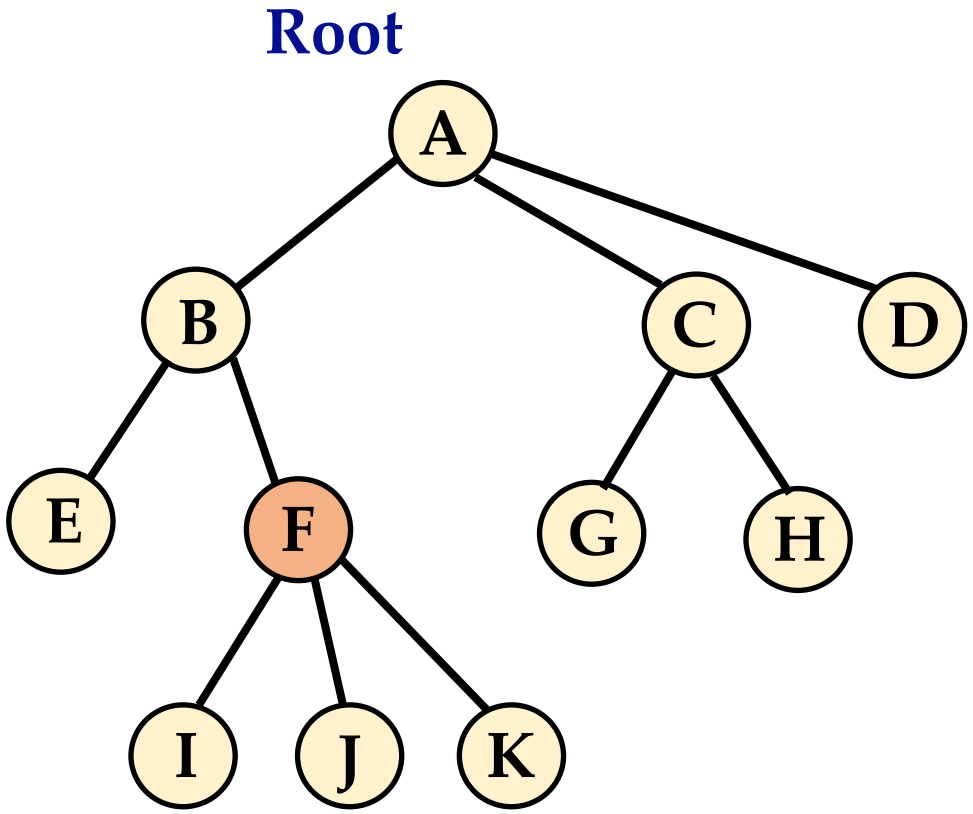


(Rooted) Tree Concepts

For a node (F)

- one parent: predecessor (B)
- (multiple) children: successors (I, J, K)
- (multiple) siblings: nodes that share the same parent (E)

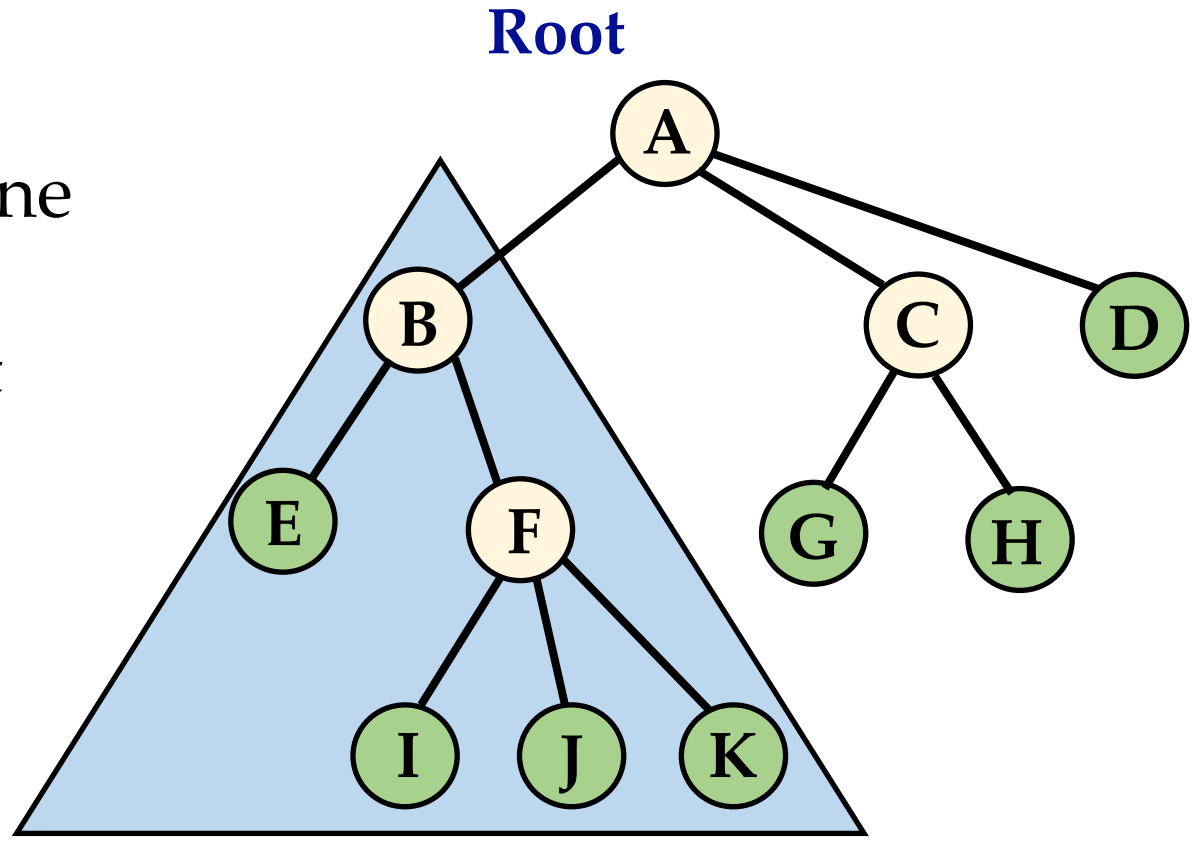
- ancestors: parent, grandparent, grand-grandparent… (B, A)
- descendant: children, grandchildren, grand-grandchildren… (I, J, K)



(Rooted) Tree Concepts

- Root: node without parent (A)
- Internal node: node with at least one child (A, B, C, F)
- External node (leaf): node without children (E, I, J, K, G, H, D)

- Subtree rooted at a node: tree consisting of a node and its descendants (subtree at B)

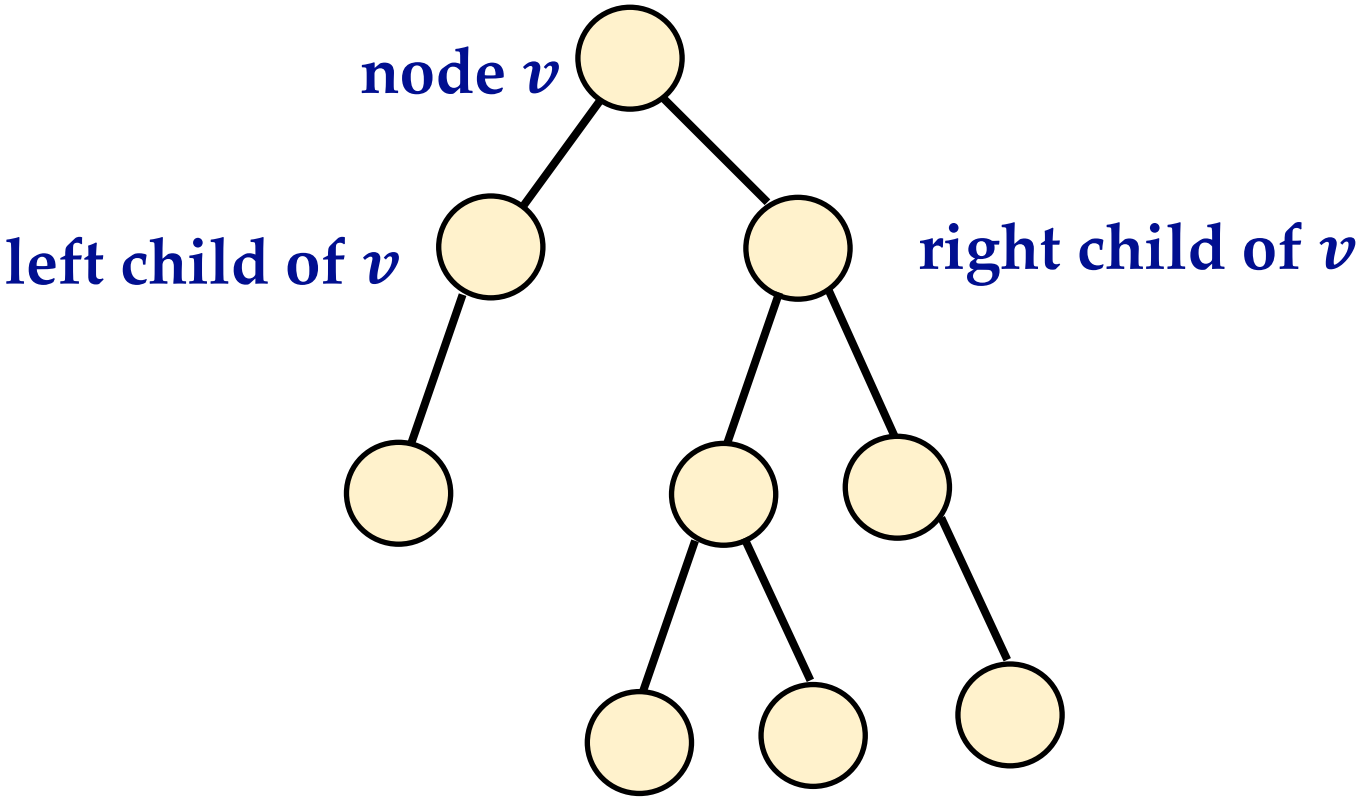


Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
- Traversals of Trees

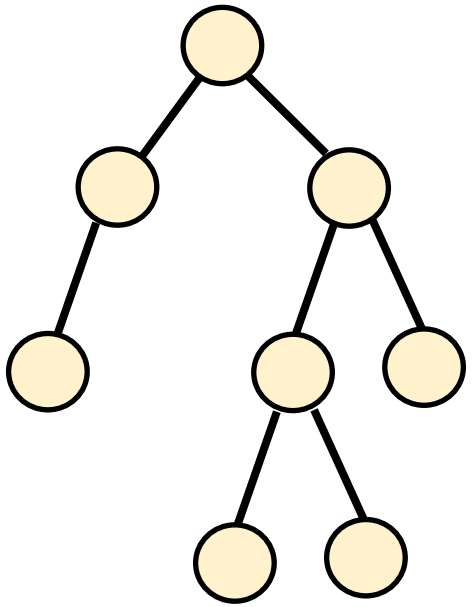
Binary Tree

- A binary tree is a rooted tree, where
  - each node has at most two children
  - children are ordered (labelled as a left or right child)

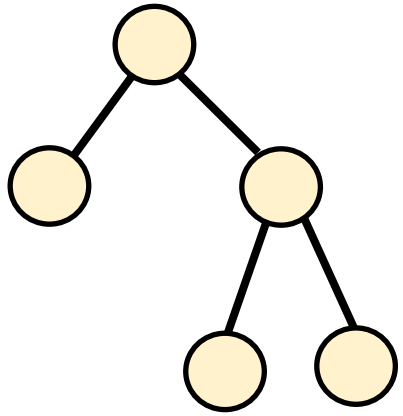


Full Binary Tree

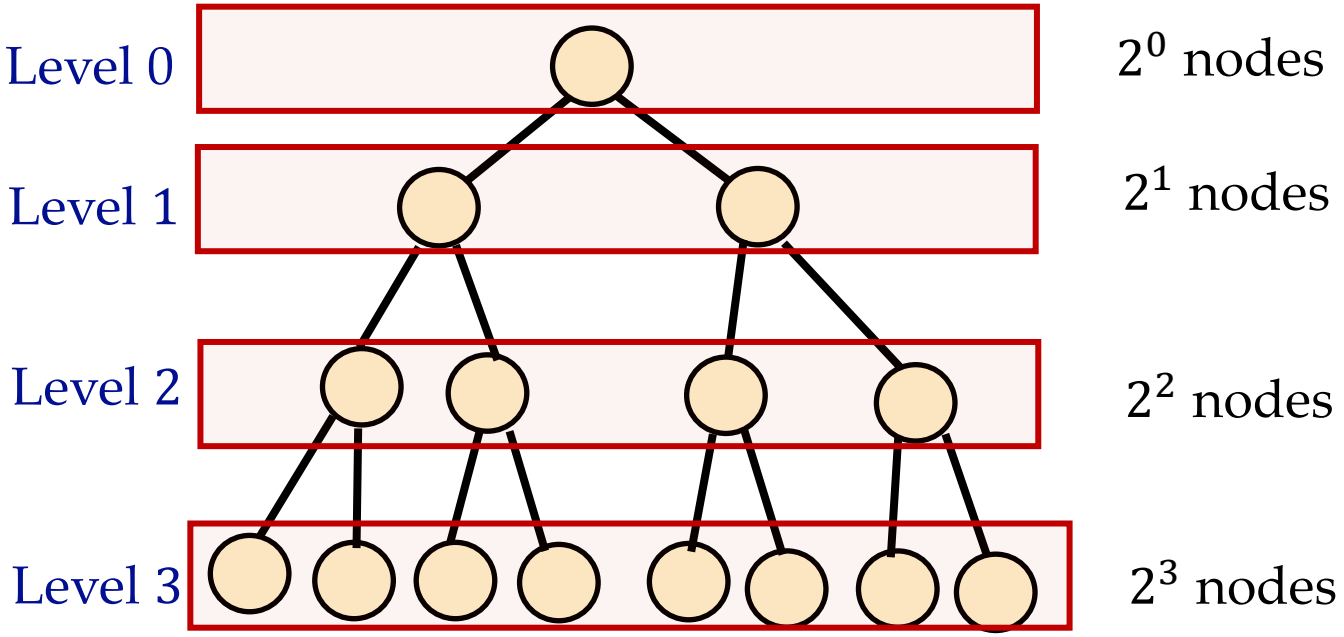
- A binary tree is full (or proper) if each node has either 0 or 2 children.
- A perfect binary tree is full with all leaves at the same level
  - level i has a total of 2^i nodes



Not full



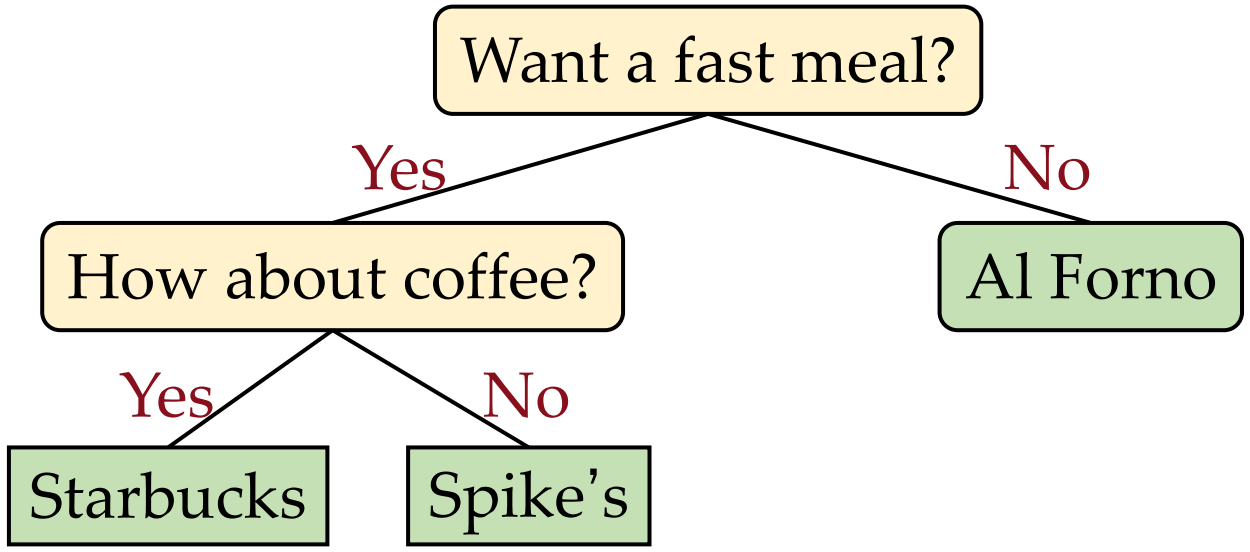
Full



Perfect

Examples of Full Binary Tree: Decision Tree

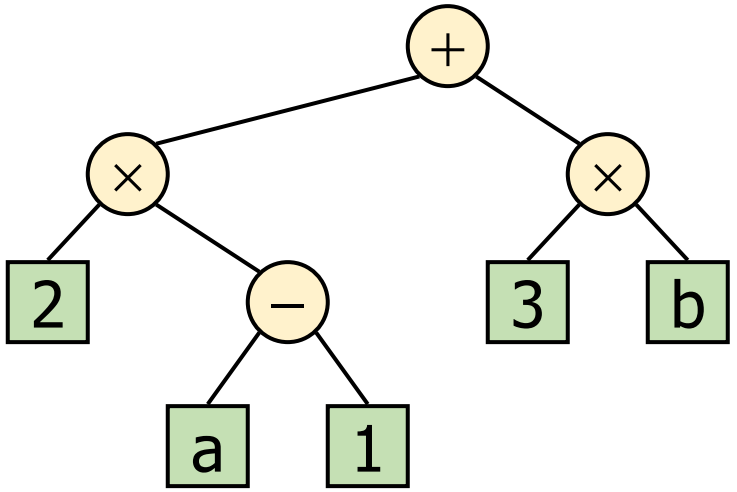
- Binary tree is associated with a decision process
  - internal nodes: questions with yes/no answer
  - external nodes: decisions



Examples of Full Binary Tree: Arithmetic Expression Tree

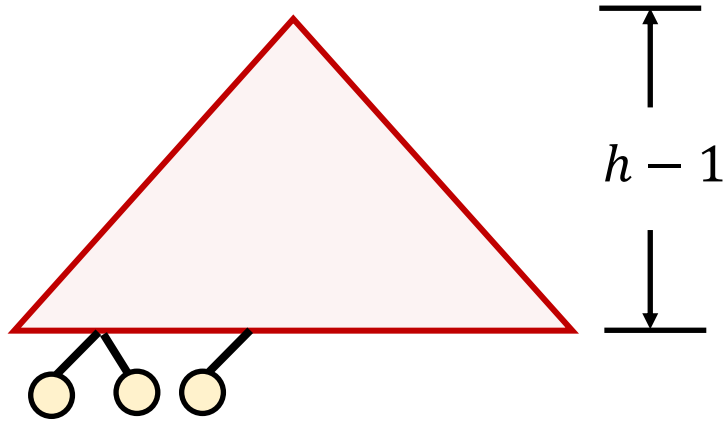
- Binary tree is associated with an arithmetic expression
  - Internal nodes: operators
  - Leaf nodes: operands

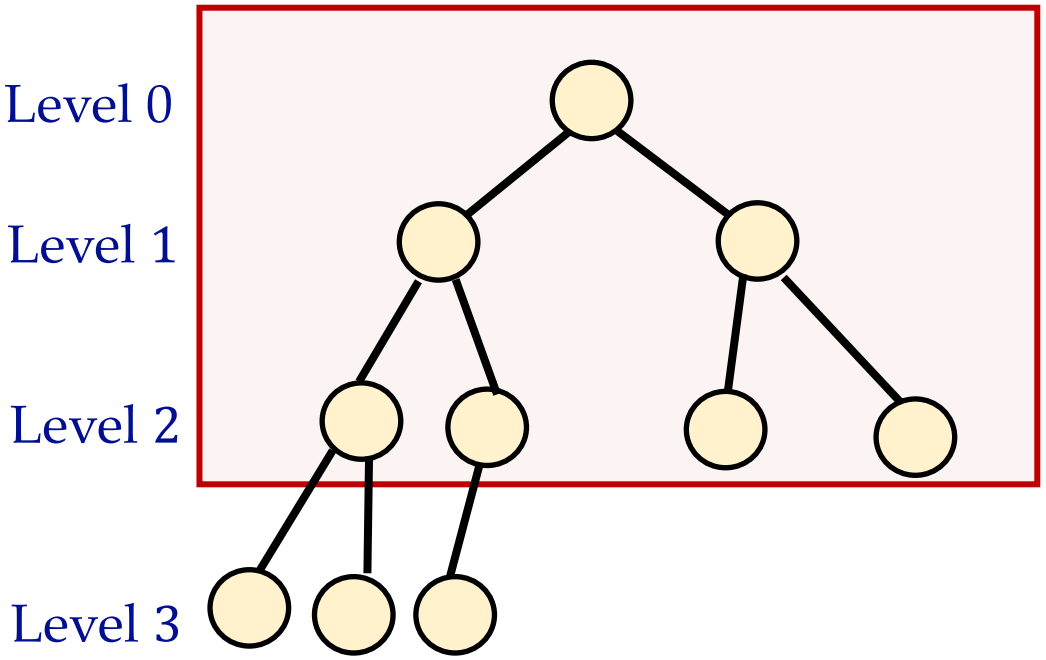
Example: arithmetic expression tree for the expression $(2 \times (a - 1) + (3 \times b))$



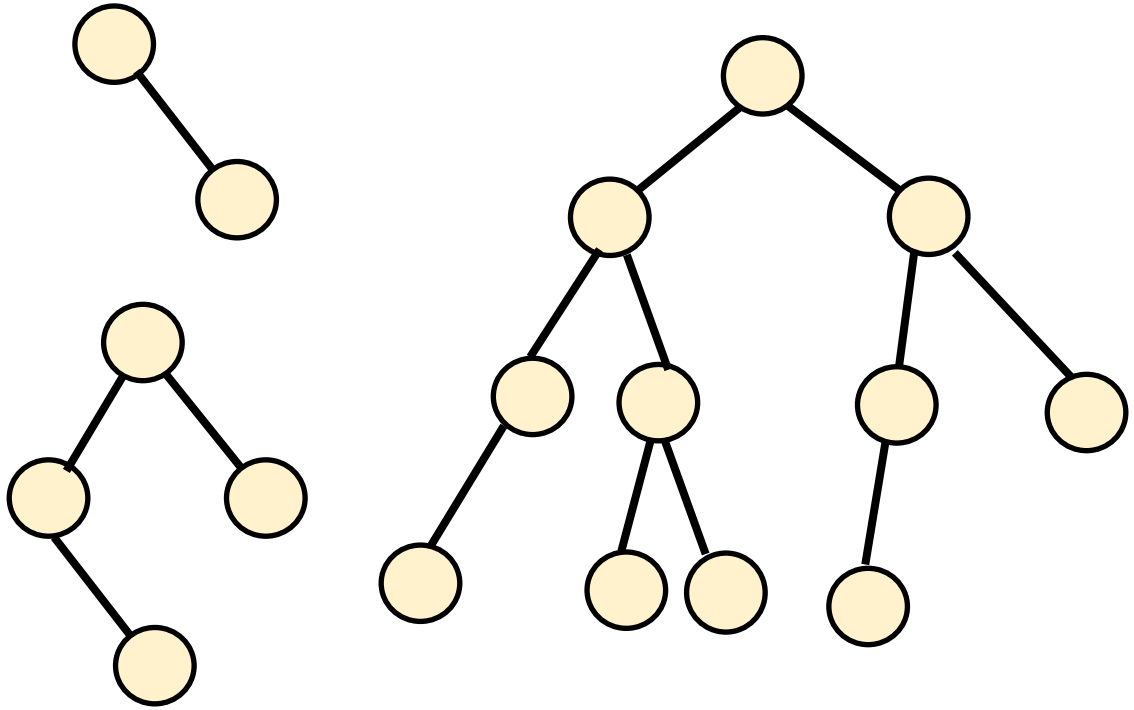
Complete Binary Tree

- A binary tree of height h is complete if
  - levels 0 to $h-1$ form a perfect tree
  - at level h, leaves are placed as left as possible





Complete



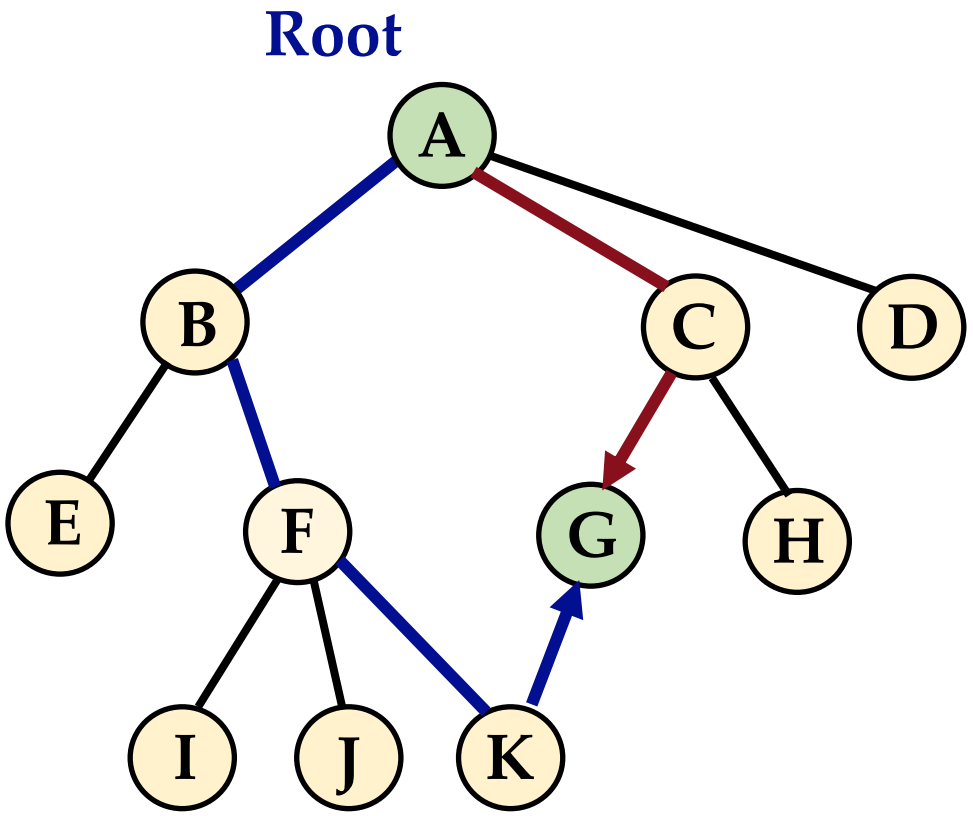
Not ccomplete

Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - binary tree property
- Traversals of Trees

Properties of Trees

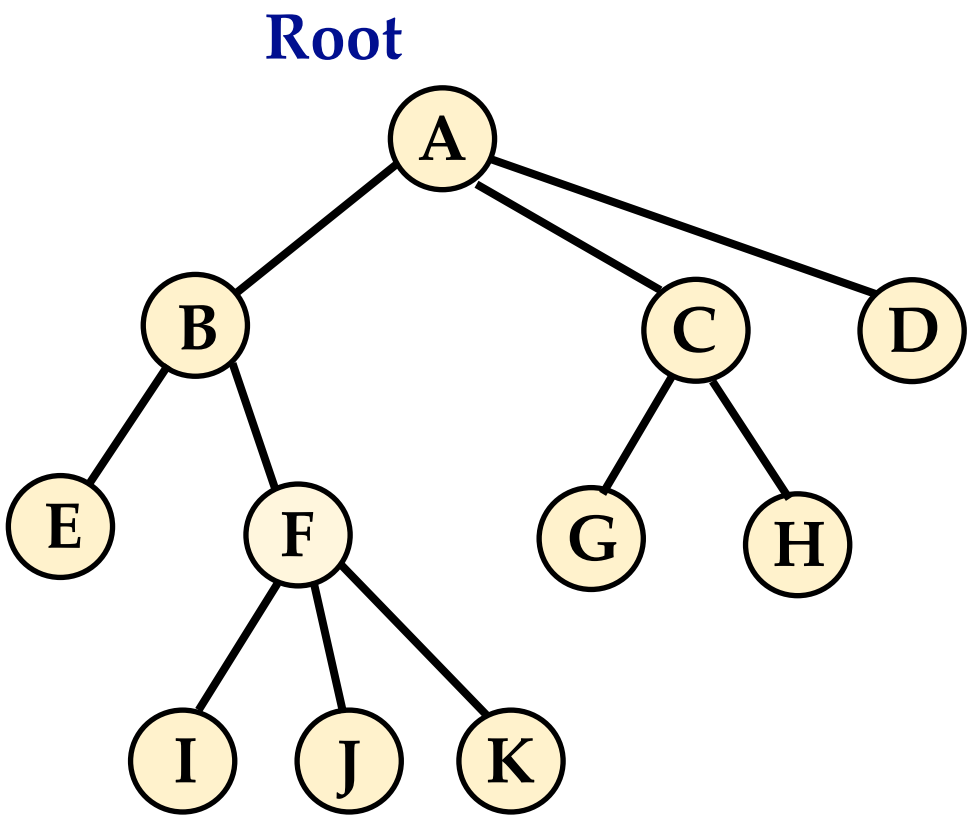
Property 1: in a tree, there exists exactly one path between any two nodes.



Suppose there exist two distinct paths between two nodes A and G. Together, these paths would form a cycle, which contradicts the definition of a tree

Properties of Trees

Property 2: a tree with n nodes has $n - 1$ edges



Every node except the root has exactly one parent edge. Therefore, there are $n - 1$ edges.

Trees and Binary Trees: roadmap

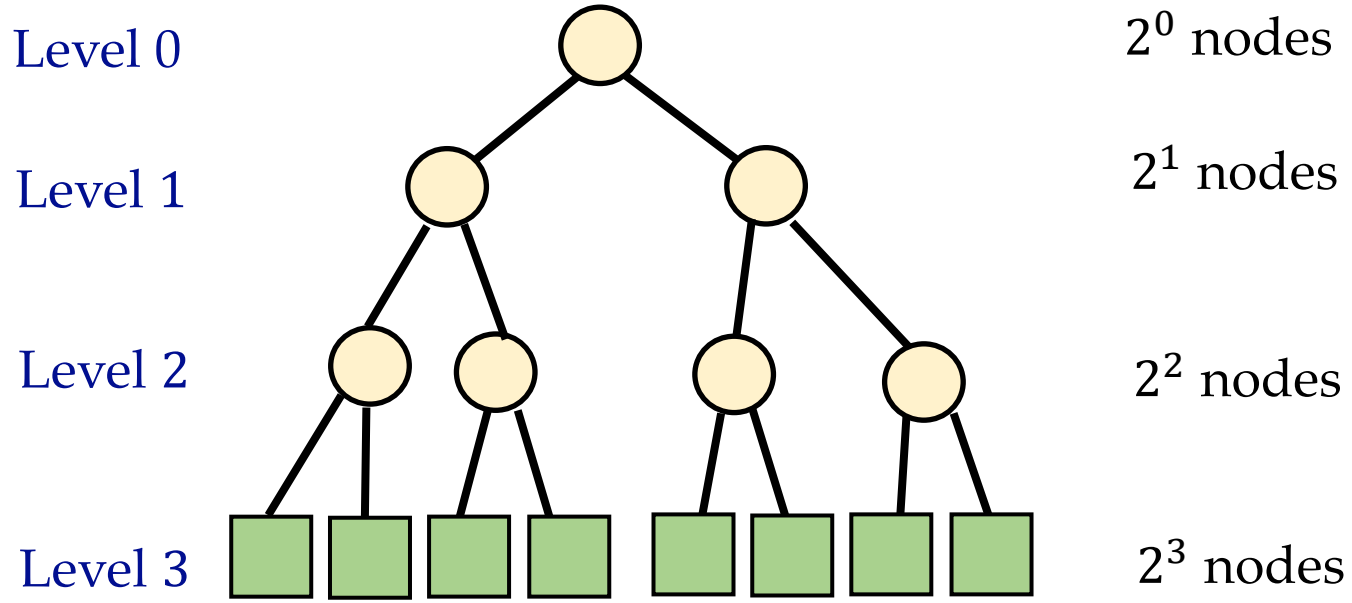
- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - Binary Tree property
- Traversals of Trees

Properties of Binary Trees

Notations:

n # of nodes n_E # of external nodes (leaves)

n_I # of internal nodes h height



Maximum number of nodes at each level ?

level i ------- 2^i

For height $h = 3$, what is the largest possible number of nodes? What is the smallest?

Perfect tree: $2^{h+1} - 1 = 15$ nodes

Single chain: 4 nodes

Properties of Binary Trees

Property 1: A non-empty binary tree has the following properties:

$h + 1 \leq n \leq 2^{h+1} - 1 \quad \Leftrightarrow \quad \log(n+1) - 1 \leq h \leq n - 1$

$1 \leq n_E \leq 2^h$

$h \leq n_{\mathrm{I}} \leq 2^h - 1$

Properties of Binary Trees

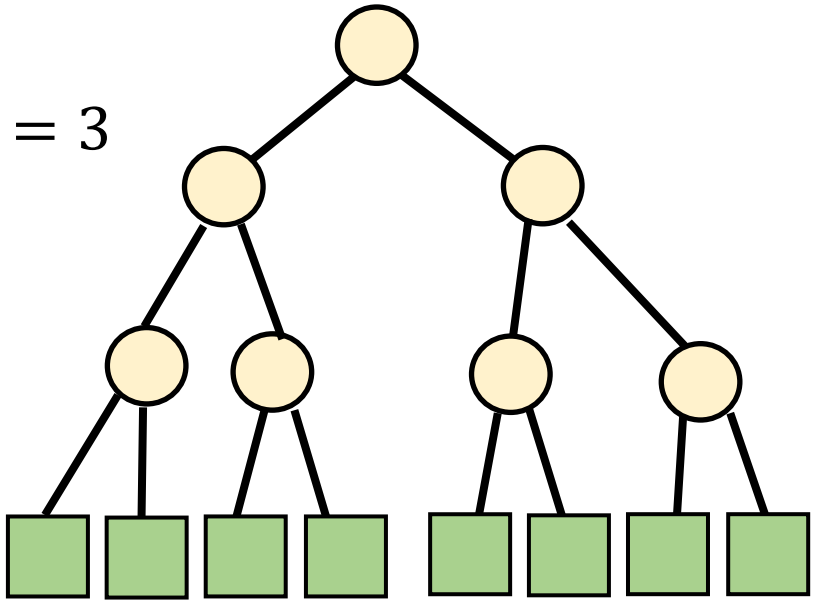
Property 1: A non-empty binary tree has the following properties:

$h + 1 \leq n \leq 2^{h+1} - 1 \quad \Leftrightarrow \quad \log(n+1) - 1 \leq h \leq n - 1$

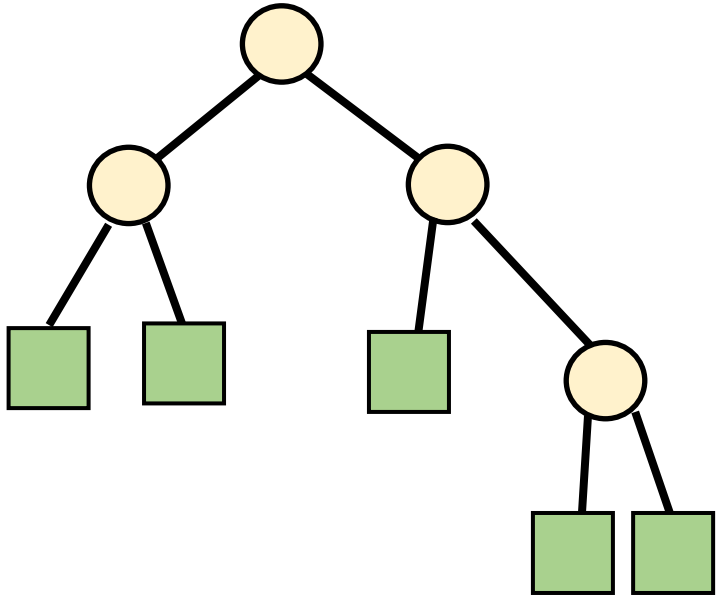
$1 \leq n_E \leq 2^h$

$h \leq n_{\mathrm{I}} \leq 2^h - 1$

$h = 3$



2^3 leaves if all at level h



Otherwise, less leaves

n_E is maximized when all leaves are at level h

$n_E \leq 2^h$

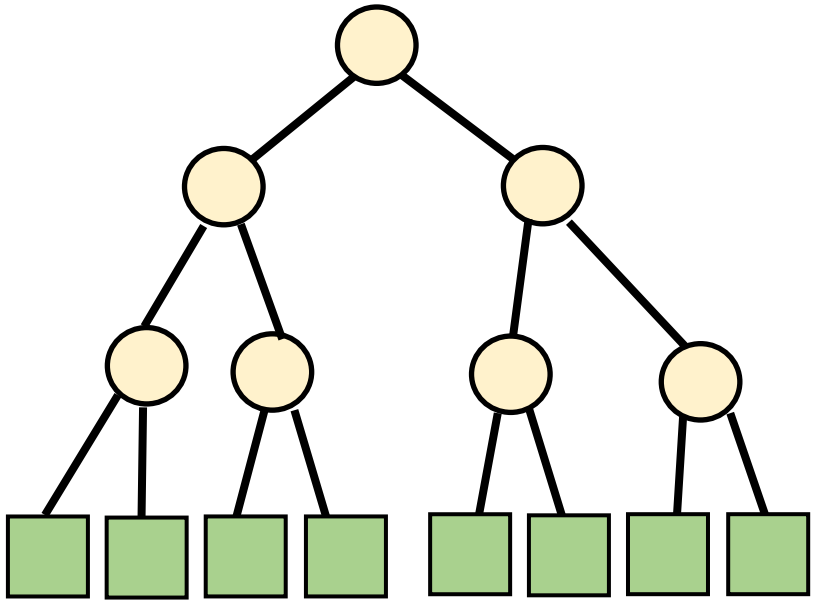
Properties of Binary Trees

Property 1: A non-empty binary tree has the following properties:

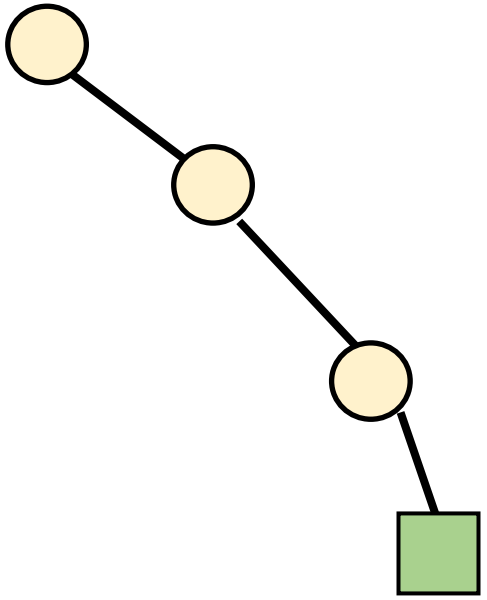
$h + 1 \leq n \leq 2^{h+1} - 1 \quad \Leftrightarrow \quad \log(n+1) - 1 \leq h \leq n - 1$

$1 \leq n_E \leq 2^h$

$h \leq n_{\mathrm{I}} \leq 2^h - 1$



$h = 3$ and $n_I = 7$



$h = 3$ and $n_I = 3$

h is the maximum number of ancestors, as there must be at least one internal node for each level (except the last)

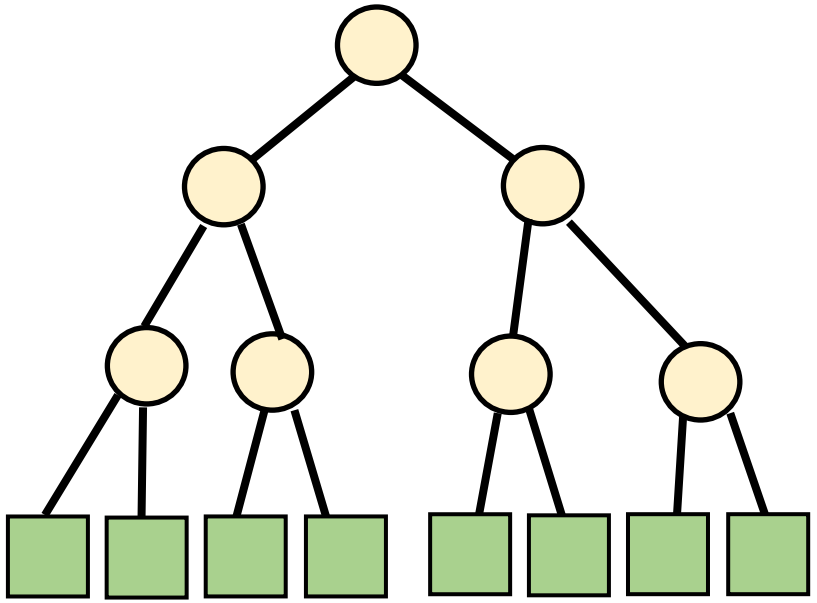
Properties of Binary Trees

Property 1: A non-empty binary tree has the following properties:

$h + 1 \leq n \leq 2^{h+1} - 1 \quad \Leftrightarrow \quad \log(n+1) - 1 \leq h \leq n - 1$

$1 \leq n_E \leq 2^h$

$h \leq n_{\mathrm{I}} \leq 2^h - 1$



$h = 3$ and $n_I = 7$

n_I is maximized when the binary tree is perfect, thus

$$n_I \leq 2^0 + 2^1 + \cdots + 2^{h-1}$$
$$\leq 2^h - 1$$

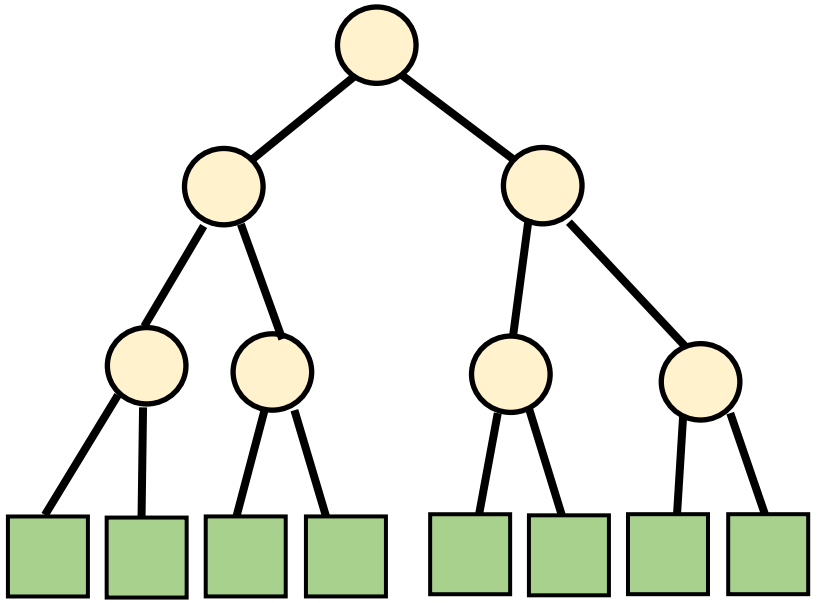
Properties of Binary Trees

Property 1: A non-empty binary tree has the following properties:

$h + 1 \leq n \leq 2^{h+1} - 1$ $\Leftrightarrow$ $\log(n+1) - 1 \leq h \leq n - 1$

$1 \leq n_E \leq 2^h$

$h \leq n_{\mathrm{I}} \leq 2^h - 1$



$h = 3$ and $n_I = 7$

$n = n_E + n_I \geq h + 1$

$n = n_E + n_I \leq 2^h + 2^h - 1 = 2^{h+1} - 1$

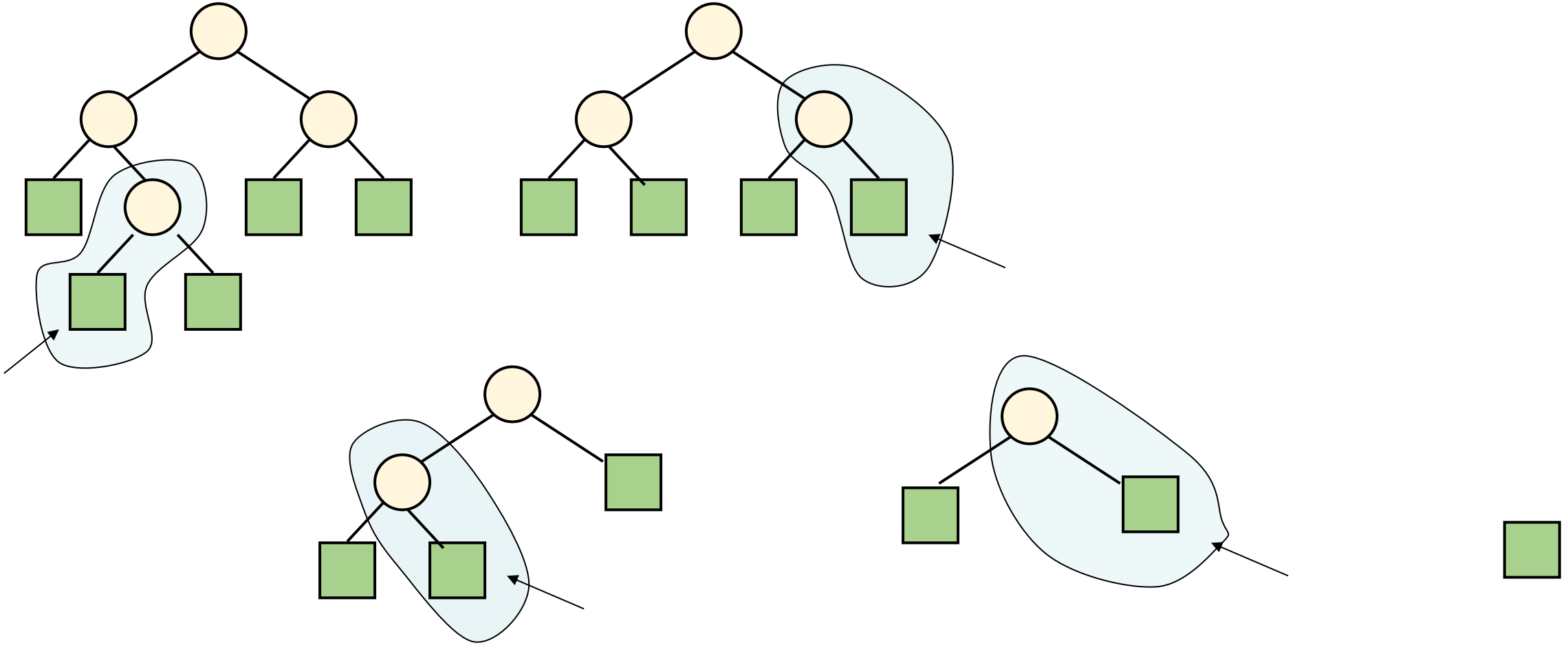
$\Leftrightarrow$

$h \leq n - 1$

$h \geq \log(n+1) - 1$

Properties of Full Binary Trees

Property 1: A non-empty full binary tree has $n_E = n_I + 1$.



Properties of Full Binary Trees

Property 1: A non-empty full binary tree has $n_E = n_I + 1$.

Since $n = n_E + n_I = 2n_E - 1$,

we have $n_E = \frac{n+1}{2} = \left\lceil \frac{n}{2} \right\rceil$,

$n_I = \frac{n-1}{2} = \left\lfloor \frac{n}{2} \right\rfloor$

Properties of Full Binary Trees

Property 2: A non-empty full binary tree has the following properties:

$2h + 1 \leq n \leq 2^{h+1} - 1 \Leftrightarrow \log(n+1) - 1 \leq h \leq (n-1)/2$

$h + 1 \leq n_E \leq 2^h$

$h \leq n_I \leq 2^h - 1$

$n_E = n_I + 1 \geq h + 1$

$n = n_E + n_I \geq h + 1 + h = 2h + 1$

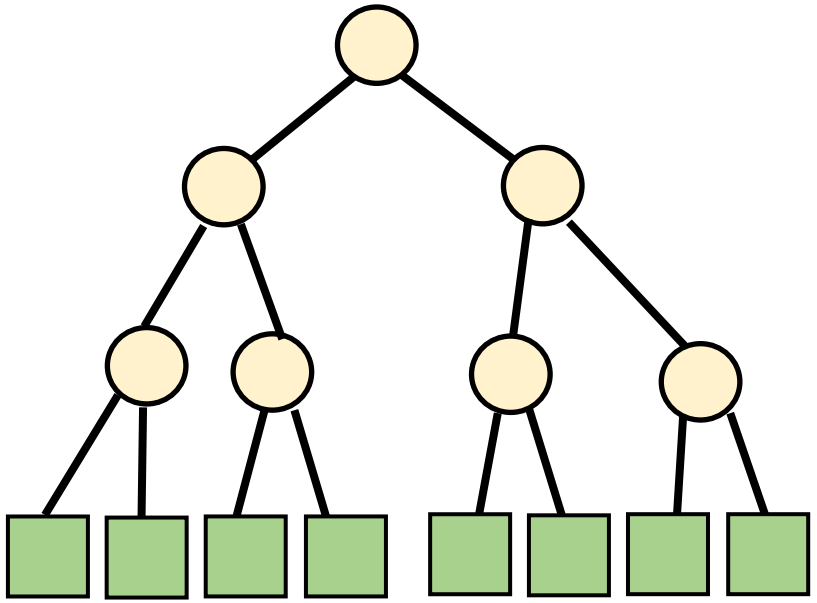
$\Leftrightarrow \quad h \leq (n-1)/2$

Having two children at every internal node does not guarantee small height.

Properties of Perfect Binary Trees

Property 1: A non-empty perfect binary tree has the following properties:

$$2h + 1 \leq n = 2^{h+1} - 1 \Leftrightarrow \log(n+1) - 1 = h$$
$$h + 1 \leq n_E = 2^h$$
$$h \leq n_I = 2^h - 1$$



$h = 3, n_I = 7, n_E = 8$

$$n_E = 2^h$$

$$n_I = 2^0 + 2^1 + \cdots + 2^{h-1} = 2^h - 1$$

$$n = 2^0 + 2^1 + \cdots + 2^h = 2^{h+1} - 1$$

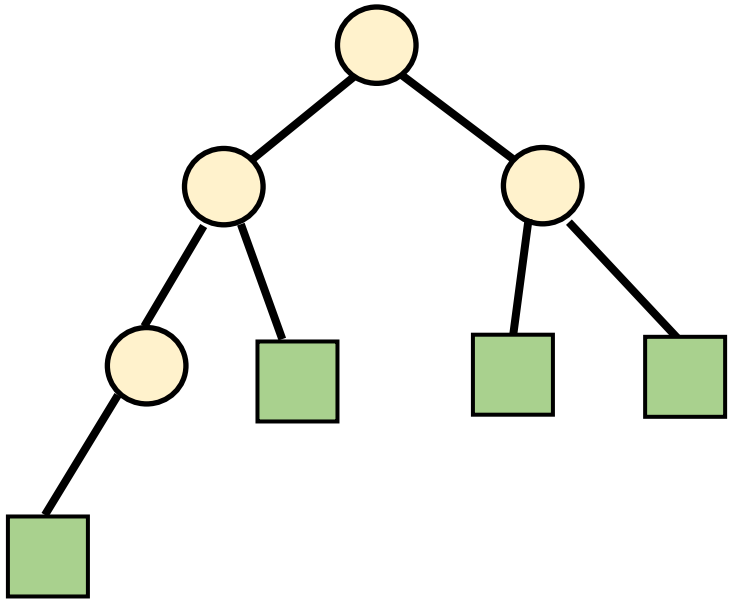
Properties of Complete Binary Trees

Property 1: A non-empty complete binary tree has the following properties:

$2^h \leq n \leq 2^{h+1} - 1 \quad \Leftrightarrow \quad h = \lfloor \log n \rfloor$

$1 \leq n_E \leq 2^h$

$h \leq n_I \leq 2^h - 1$



$h = 3, n_I = 4, n_E = 4$

A complete binary tree of height h is composed by a perfect binary tree of height h-1 plus some leaves.

$$n \geq 2^0 + 2^1 + \cdots + 2^{h-1} + 1 = 2^h$$

$$n \leq 2^{h+1} - 1 < 2^{h+1}$$

$\Leftrightarrow \quad h \leq \log n < h + 1$

h is the integer part of $\log n \Rightarrow \lfloor \log n \rfloor$

Summary

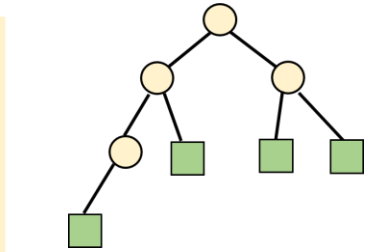
Binary tree

$h + 1 \le n \le 2^{h+1} - 1$

$1 \le n_E \le 2^h$

$h \le n_{\mathrm{I}} \le 2^h - 1$

$\log(n+1) - 1 \le h \le n - 1$



Complete binary tree

$2^h \le n \le 2^{h+1} - 1$

$1 \le n_E \le 2^h$

$h \le n_{\mathrm{I}} \le 2^h - 1$

$h = \lfloor \log n \rfloor$

$n_E = n_I + 1$

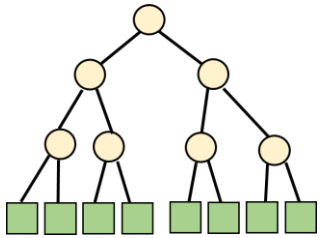
Full binary tree

$2h + 1 \le n \le 2^{h+1} - 1$

$h + 1 \le n_E \le 2^h$

$h \le n_{\mathrm{I}} \le 2^h - 1$

$\log(n+1) - 1 \le h \le (n-1)/2$



Perfect binary tree

$2h + 1 \le n = 2^{h+1} - 1$

$h + 1 \le n_E = 2^h$

$h \le n_{\mathrm{I}} = 2^h - 1$

$h = \log(n+1) - 1$

Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - Binary Tree property
- Traversals of Trees
  - Tree traversals
  - Binary tree traversals
- Implementing Tree ADT

Tree ADT

Accessor methods

- root(): return the position of the root of the tree (or null if empty)
- parent(p): return the position of the parent of position p (or null if p is the root)
- children(p): return the positions of all the children of position p

Query methods

- isRoot(p), isInternal(p), isExternal(p)

Positional container methods

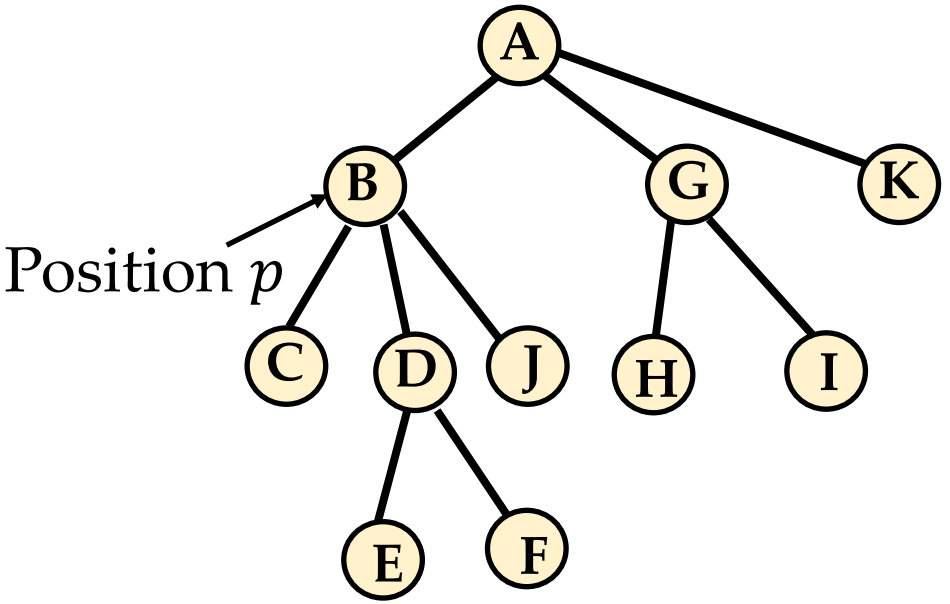
- swapElements(p, q), replaceElement(p, e), positions()

Support methods

- size(), isEmpty(), elements()

Update methods

- application specific



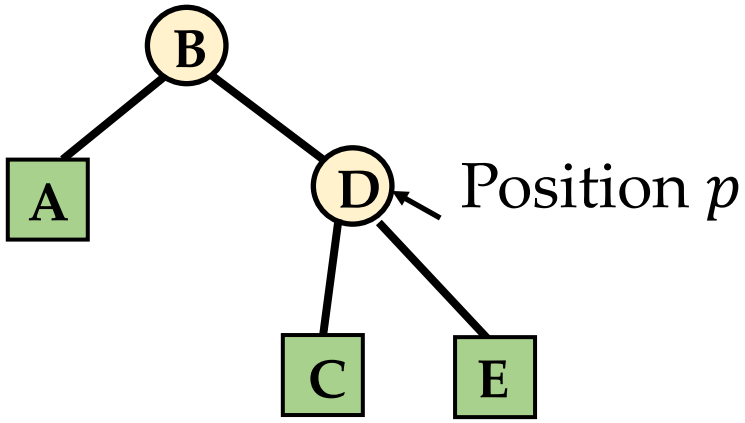
Binary Tree ADT extends Tree ADT

Additional accessor methods

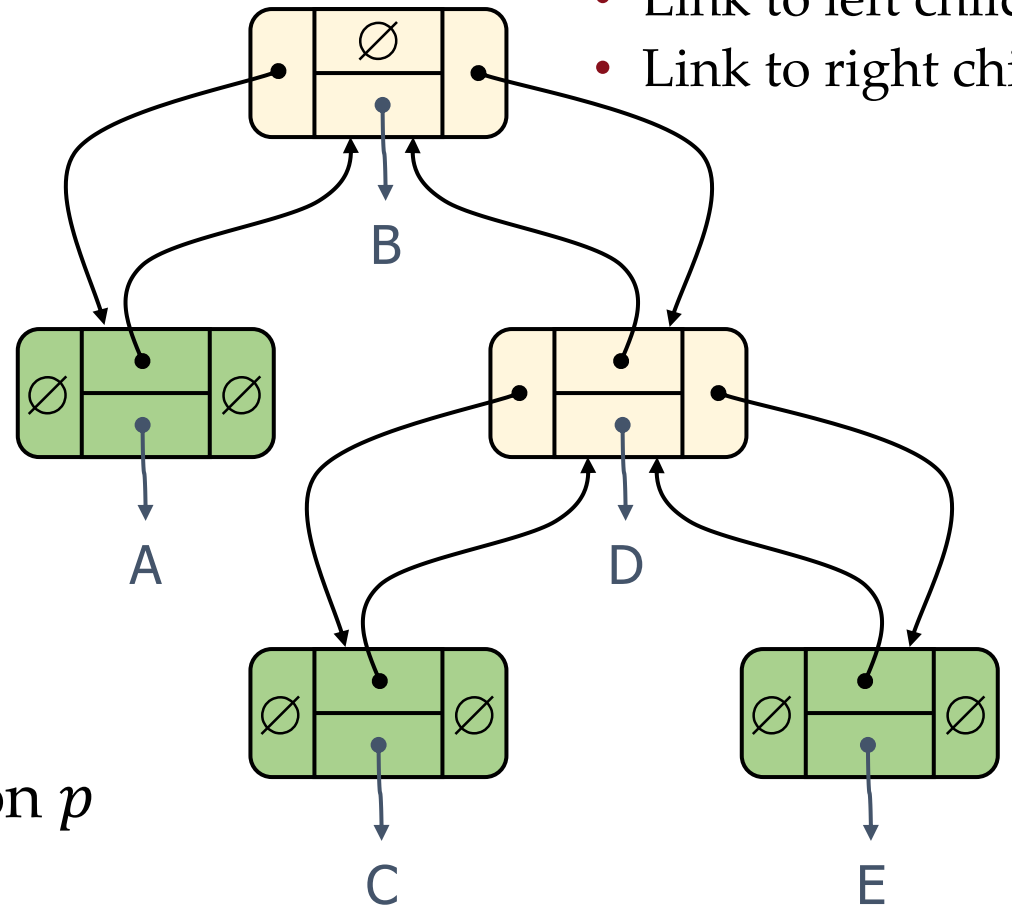
- leftChild(p), rightChild(p), sibling(p):

Update methods

- expandExternal(p)
- removeAboveExternal(p)
- Other application specific methods



- Each node contains
  - Element
  - Link to parent node
  - Link to left child node
  - Link to right child node

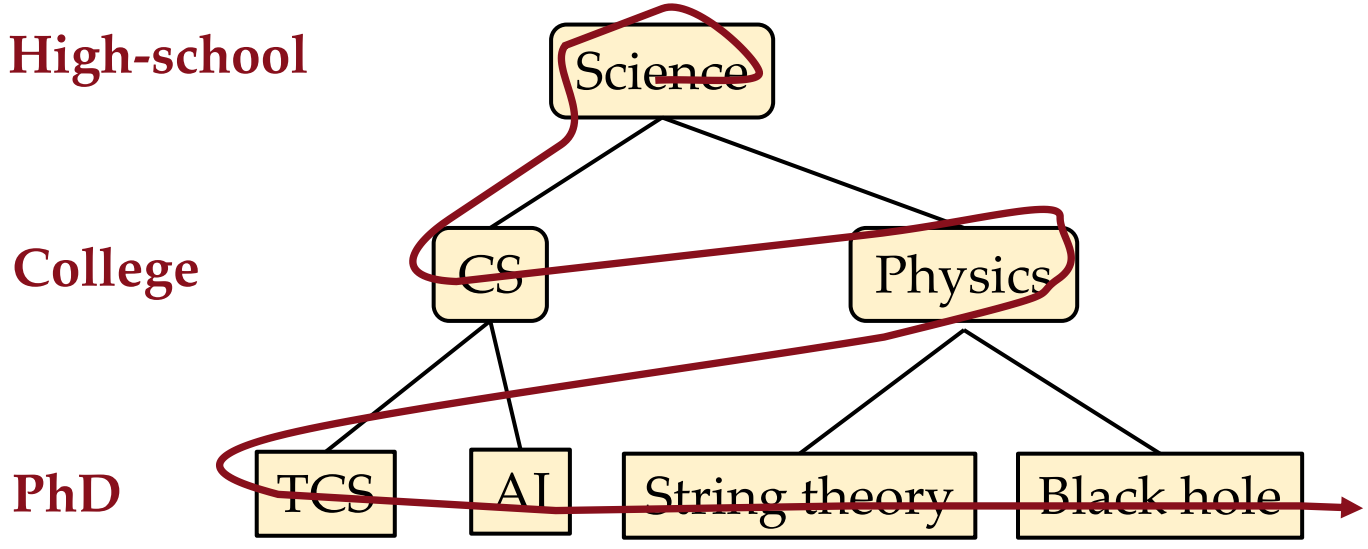


Implementation using a linked structure

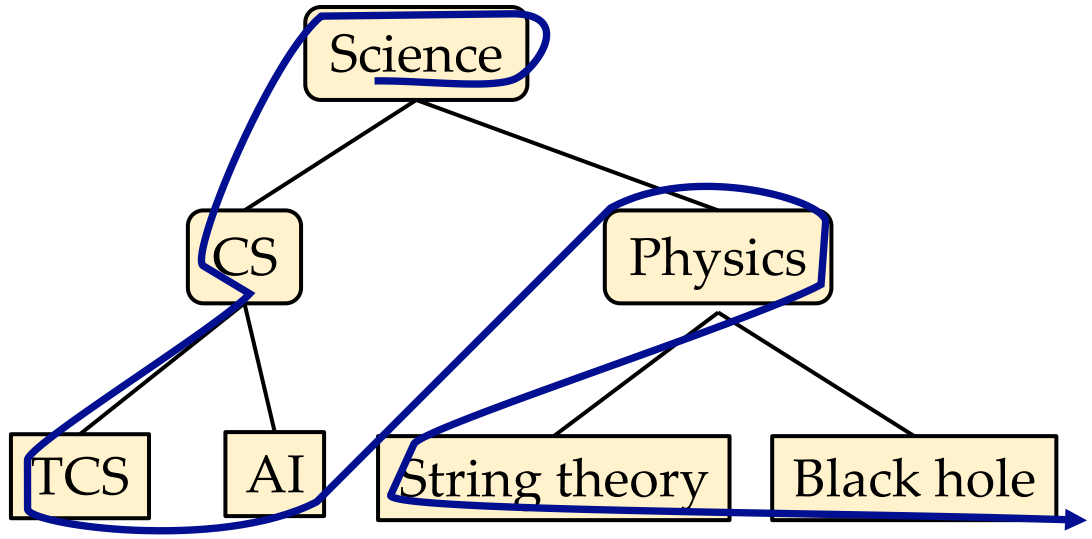
Traversal of Trees

- A tree traversal visits each node of a tree exactly once in some order

Example: Path to PhD



Breadth-first: learn as many subjects as possible at each stage



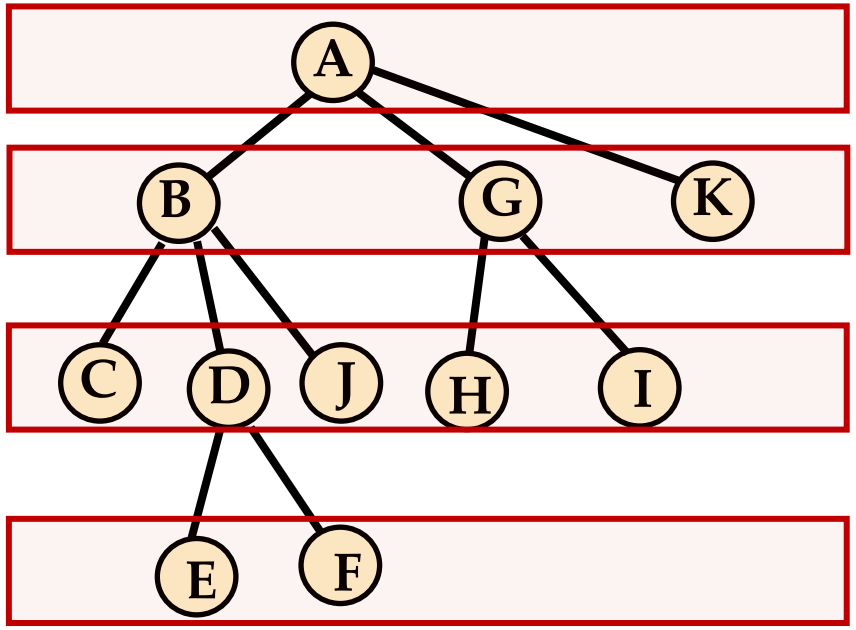
Depth-first: go as deep as possible in a specific subject

Traversal of Trees

- A tree traversal visits each node of a tree exactly once in some order

- Breadth-first traversal
  - level by level

(A, B, G, K, C, D, J, H, I, E, F)



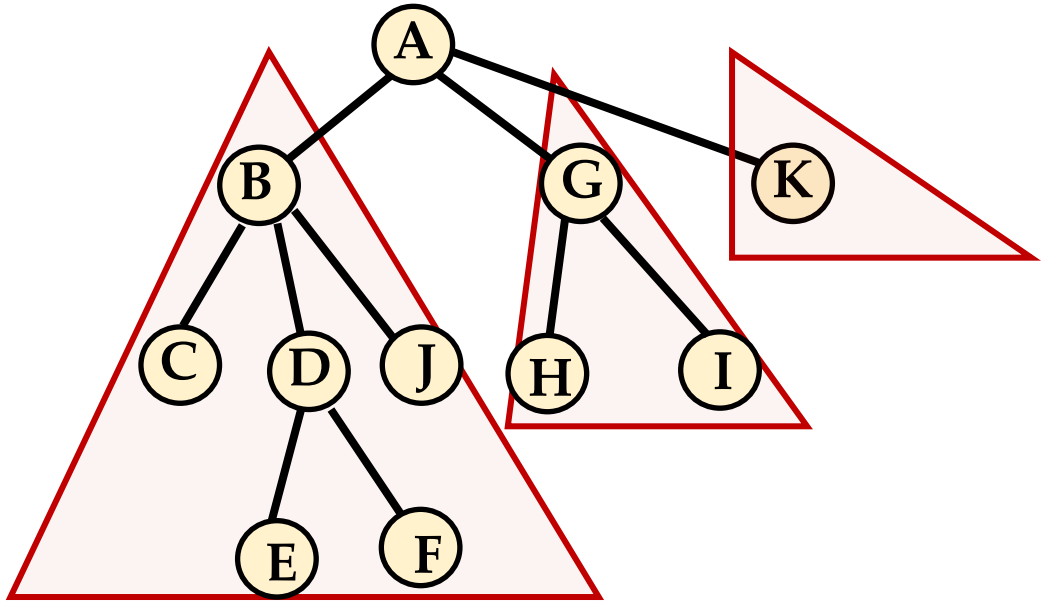
Traversal of Trees

- A tree traversal visits each node of a tree exactly once in some order

- Breadth-first traversal
  - level by level

  (A, B, G, K, C, D, J, H, I, E, F)

- Depth-first traversal
  - Visiting the root and the subtrees rooted at its children, recursively.
  - Preorder traversal: root first then subtrees
  - Postorder traversal: subtrees first then root

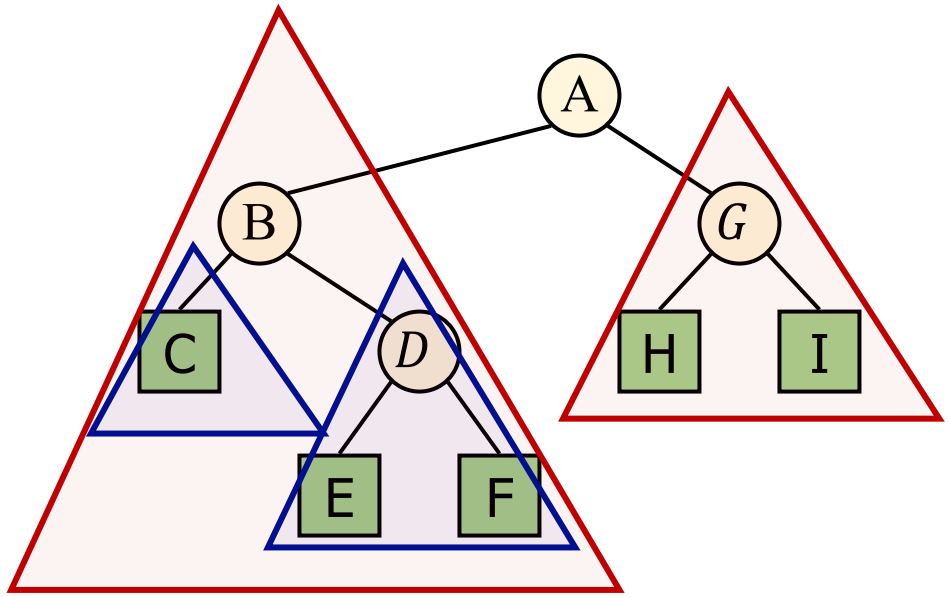


Traversing Trees: Preorder

- Preorder traversal: root first then subtrees (its descendants)
- Recursive algorithm

```
Algorithm preOrder(v)
If v is external then
    visit v
Else
    visit v
    For each child w of v do
        preOrder(w)
```

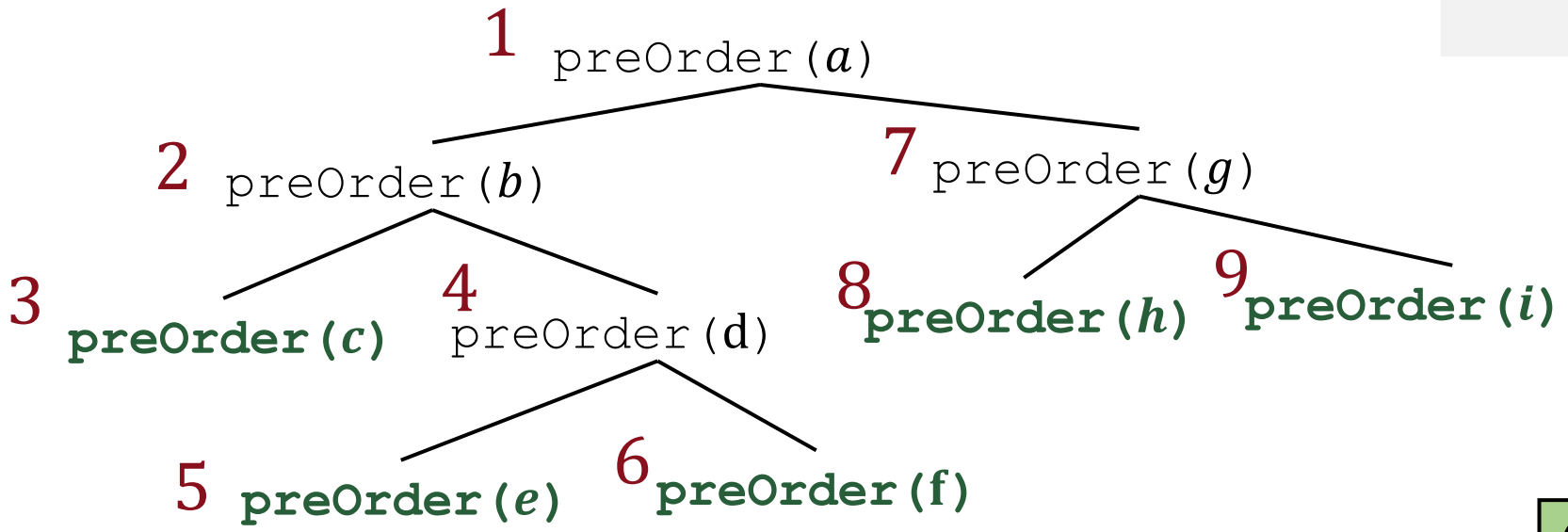
`preOrder(w)` visits every node in the subtree rooted at *w*, in preorder.



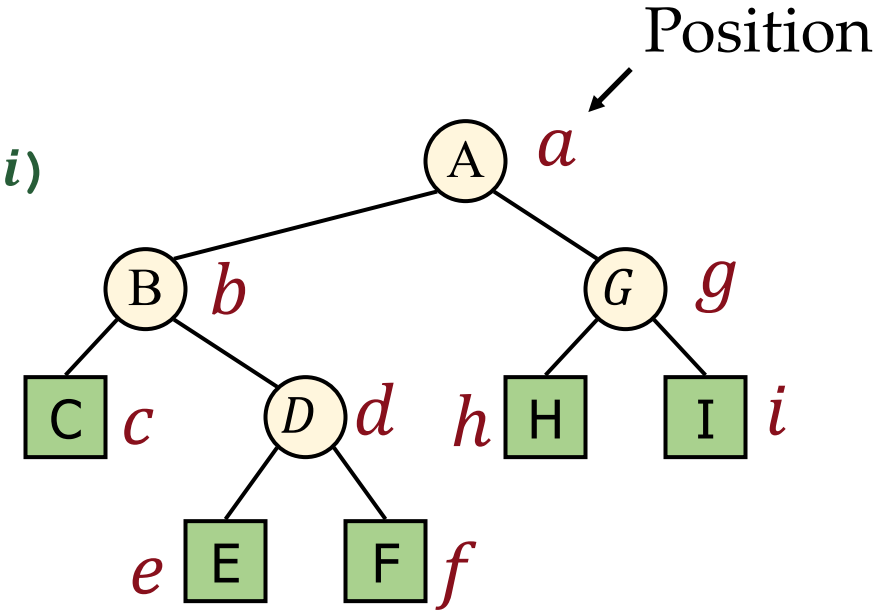
Traversing Trees: Preorder

- Preorder traversal: root first then subtrees (its descendants)
- Recursive algorithm

```
Algorithm preOrder(v)
    visit v
    If v is internal then
        For each child w of v do
            preOrder(w)
```



Trace of recursive calls

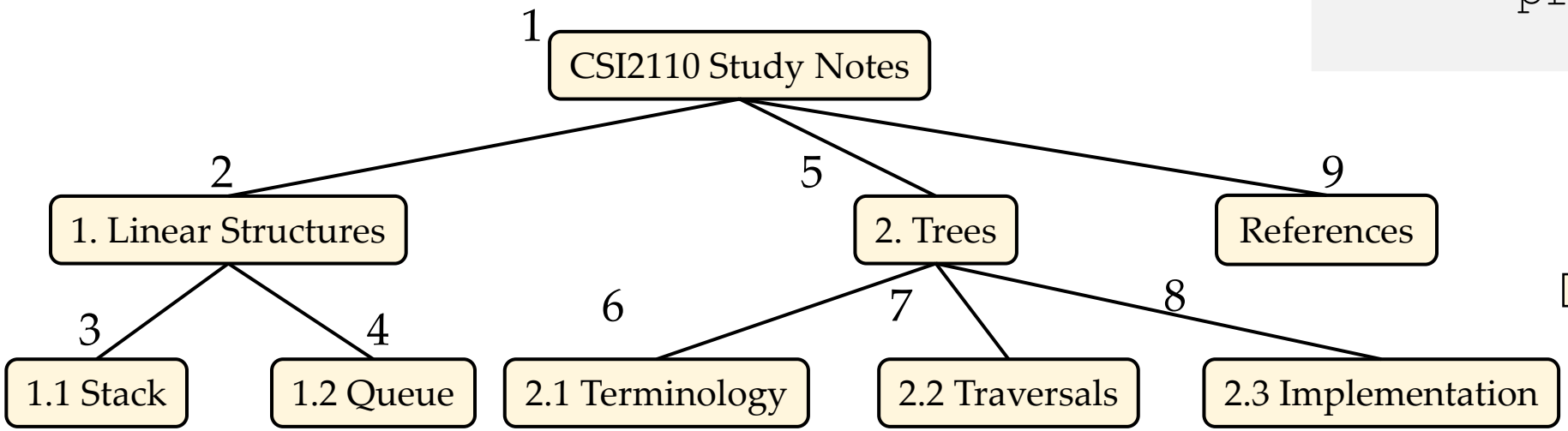


Output: A, B, C, D, E, F, G, H, I

Traversing Trees: Preorder

- Application: print a structured document

```
Algorithm printDocu(v,level)
  printIndent(level)
  print(v.element)
  If v is internal then
     For each child w of v do
        printDocu(w,level+1)
```

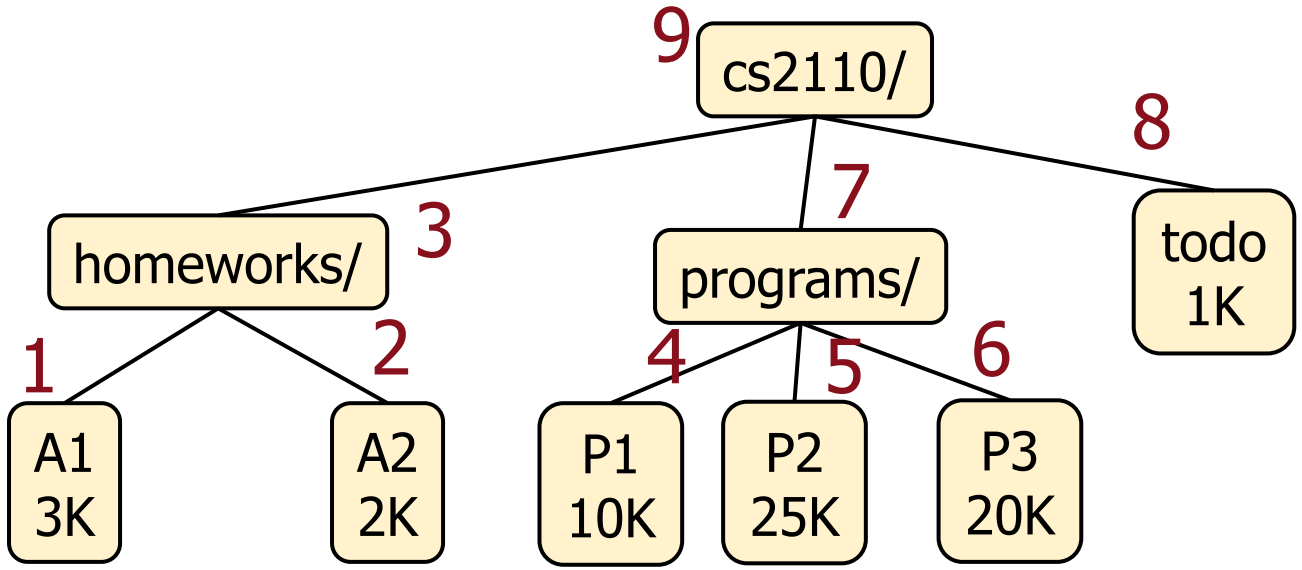


CSI2110 Study Notes
1. Linear Structures
   1.1 Stack
   1.2 Queue
2. Trees
   2.1 Terminology
   2.2 Traversals
   2.3 Implementation

References

Traversing Trees: Postorder

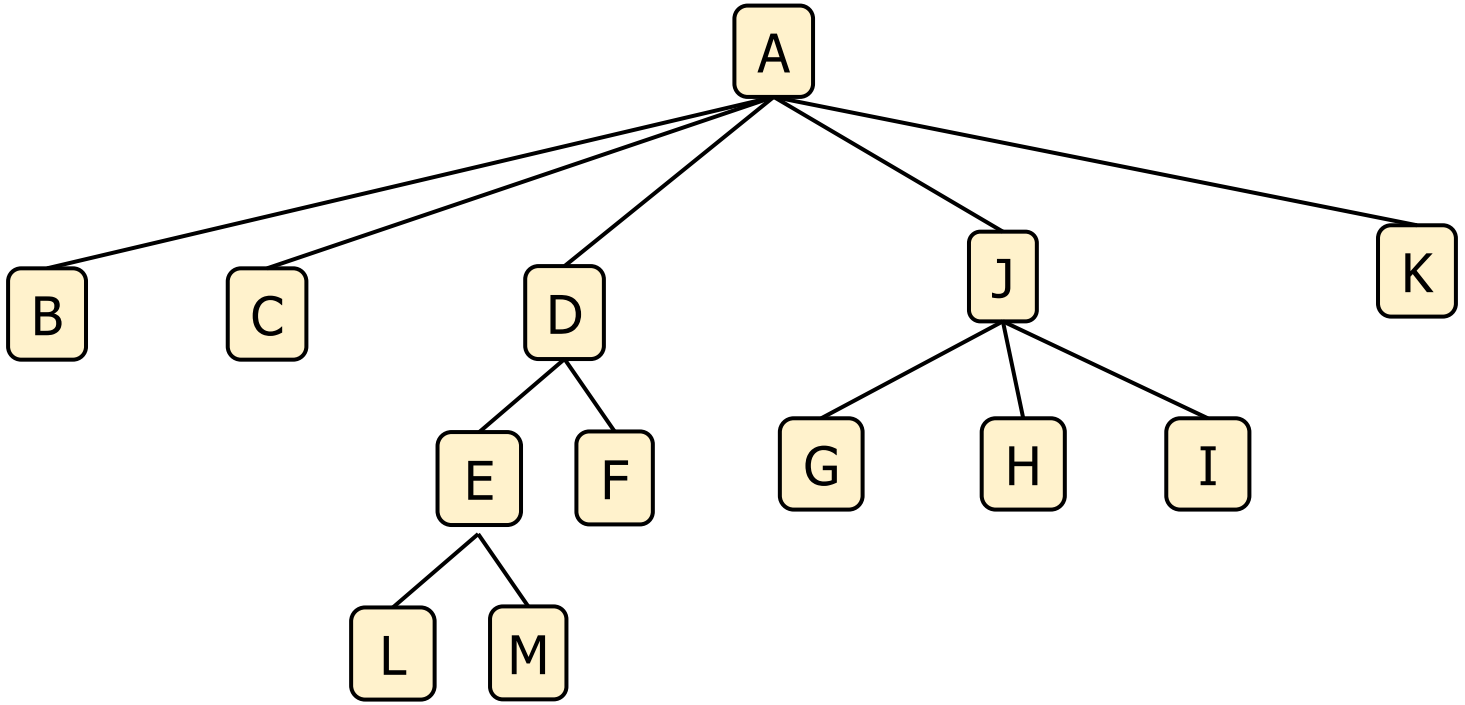
- Postorder traversal: subtrees (descendants) first then root
- Application: compute space used by files in a directory and its subdirectories




```
Algorithm postOrder(v)
 If v is internal then
    For each child w of v do
        postOrder(w)
 visit v
```

```
Algorithm dirSize(v)
 If v is internal then
    total ← 0
    For each child w of v do
      total ← total + dirSize(w)
    return total
 return v.size
```

Exercise



Breadth-first traversal: A, B, C, D, J, K, E, F, G, H, I, L, M

Preorder traversal: A, B, C, D, E, L, M, F, J, G, H, I, K

Postorder traversal: B, C, L, M, E, F, D, G, H, I, J, K, A

Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - Binary Tree property
- Traversals of Trees
  - Tree traversals
  - Binary tree traversals
- Implementing Tree ADT

Traversal of Binary Trees

- Breadth-first traversal
  - level by level

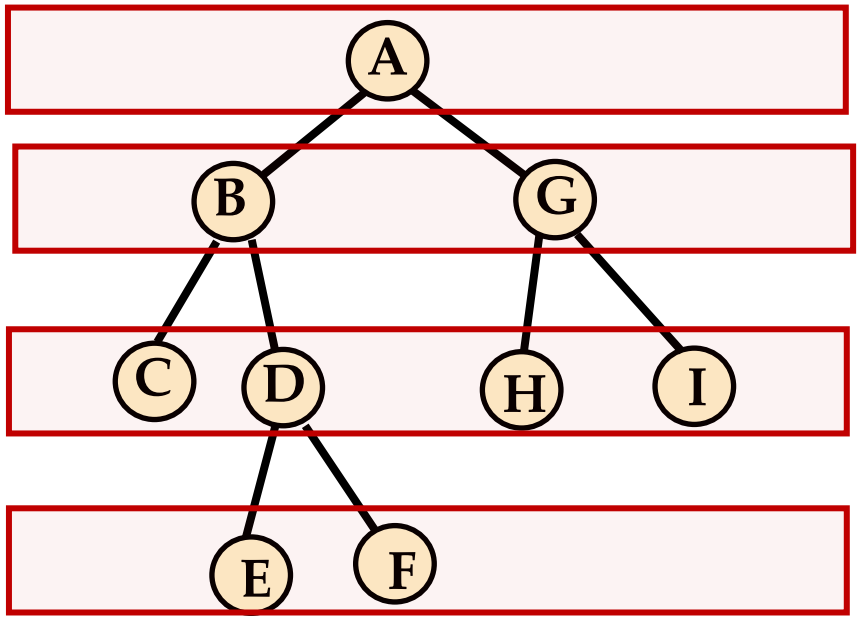
  (A, B, G, C, D, H, I, E, F)

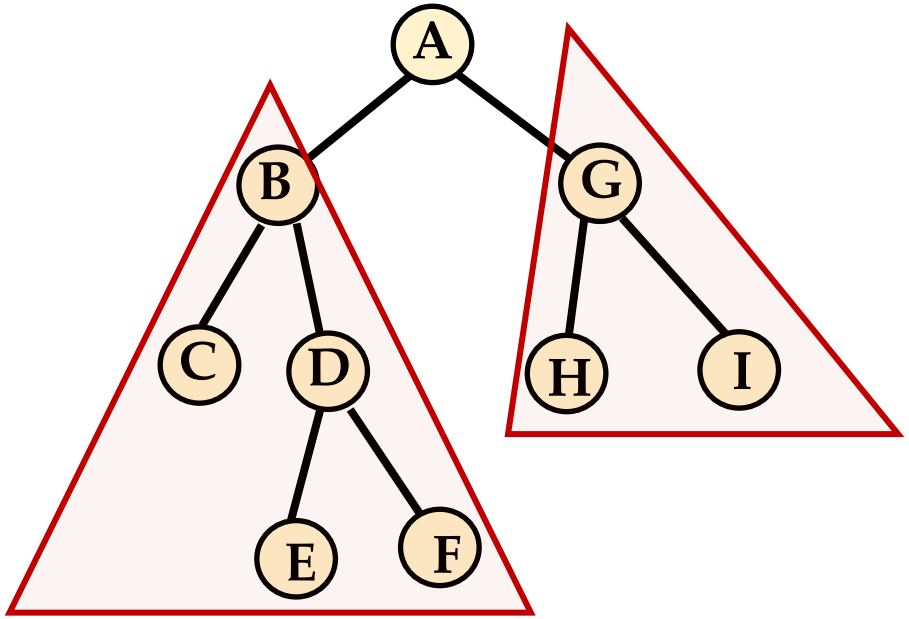
- Depth-first traversal
  - Preorder traversal: root → left subtree → right subtree

    (A, B, C, D, E, F, G, H, I)
  - Postorder traversal: left subtree → right subtree → root

    (C, E, F, D, B, H, I, G, A)
  - Inorder traversal: left subtree → root → right subtree

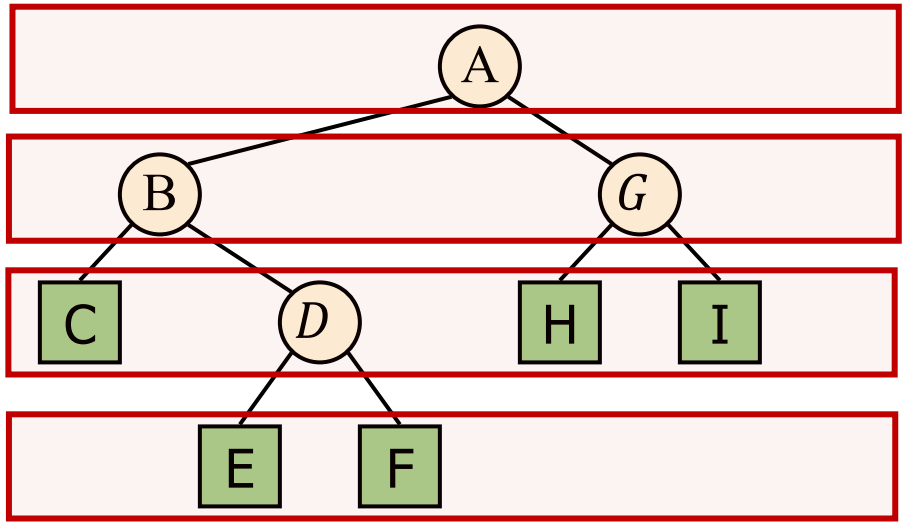
    (C, B, E, D, F, A, H, G, I)



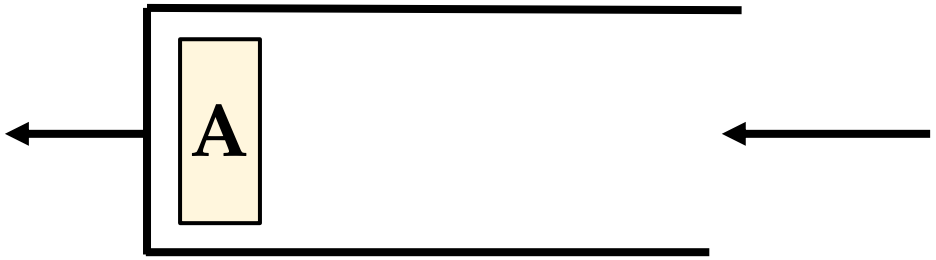


Binary tree Traversal: Breadth-first traversal

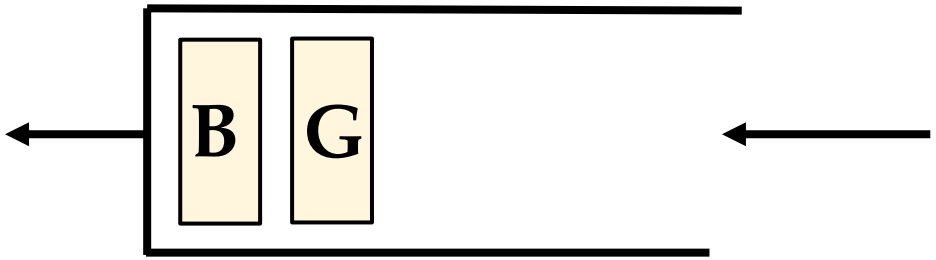
- When visiting each node, store all its children's positions
  - using a queue (FIFO)



enqueue *A*



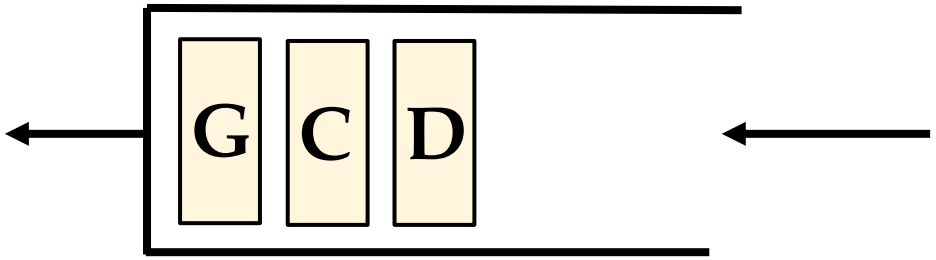
Dequeue A, visit *A* and enqueue *A*'s children

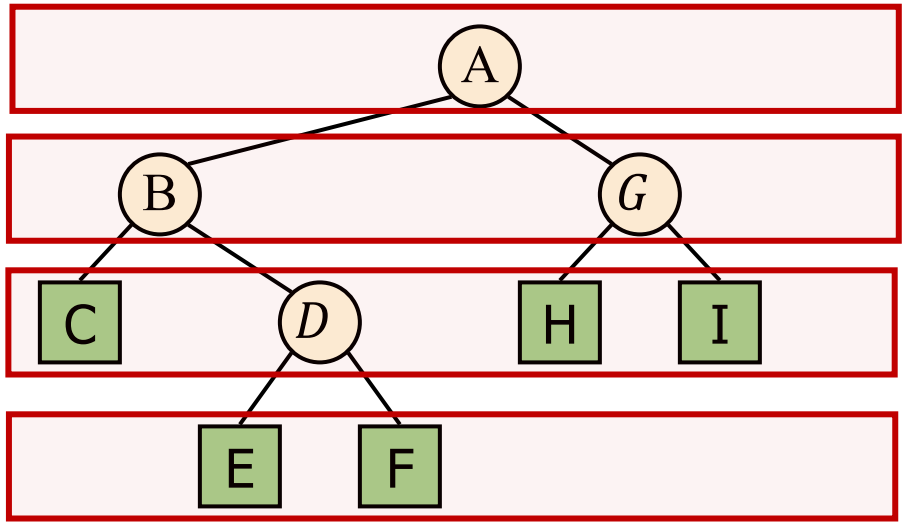


Binary tree Traversal: Breadth-first traversal

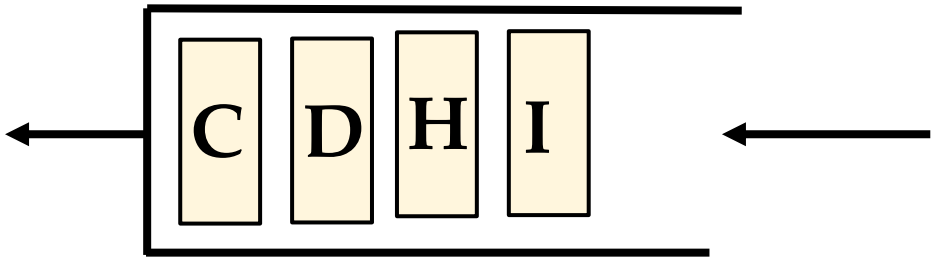
- When visiting each node, store all its children's positions
  - using a queue (FIFO)

dequeue *B*, visit *B*, and enqueue *B*'s children





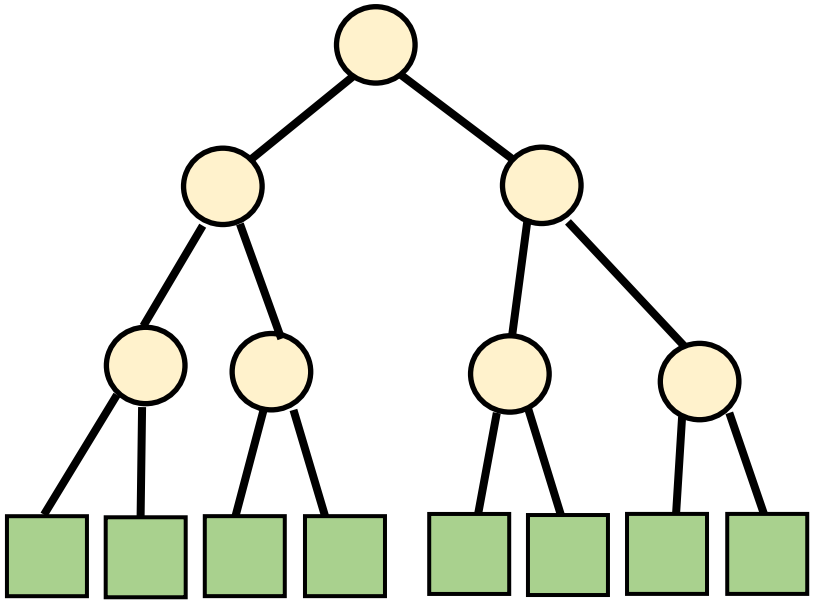
Dequeue G, visit G and enqueue G's children

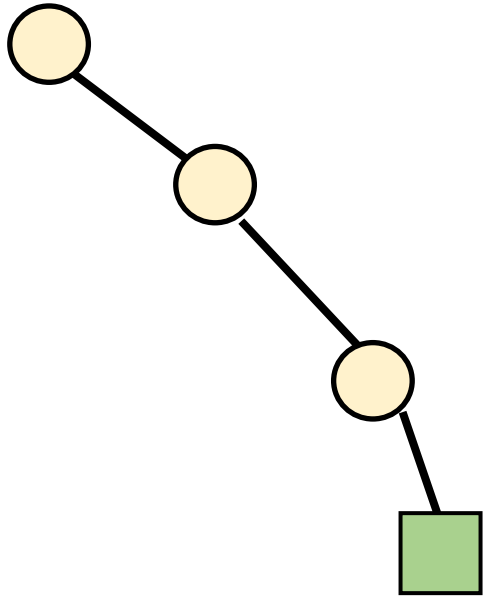


Breadth-first traversal:
A, B, G, C, D, H, I, E,F

Breadth-first traversal: Performance

- Time efficiency: $O(n)$
- Space efficiency: the size of queue is maximized when the tree is perfect, and the maximal size is the number of leaves





Worst case: $O(n_E) = O\left(\left\lceil \frac{n}{2} \right\rceil\right) = O(n)$

Best case: $O(1)$

Depth-first traversal algorithms

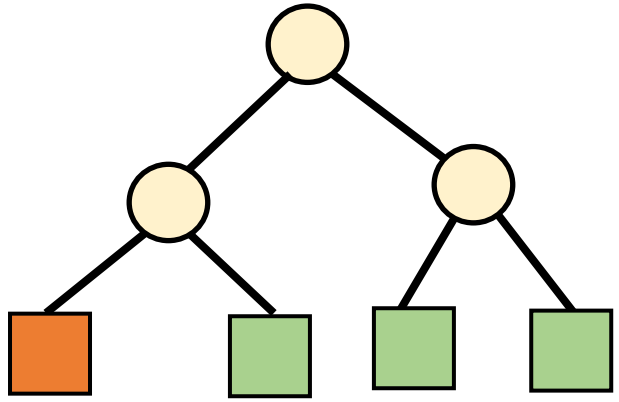
```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

```
Algorithm postOrder(v)
   If v is internal then
      postOrder(leftChild(v))
      postOrder(rightChild(v))
   visit v
```

```
Algorithm inOrder(v)
  If v is internal then
     inOrder(leftChild(v))
  visit v
  If v is internal then
     inOrder(rightChild(v))
```

One implementation detail

```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```



The previous algorithms assume that every internal node has two children. For a general binary tree, we must also handle missing children.

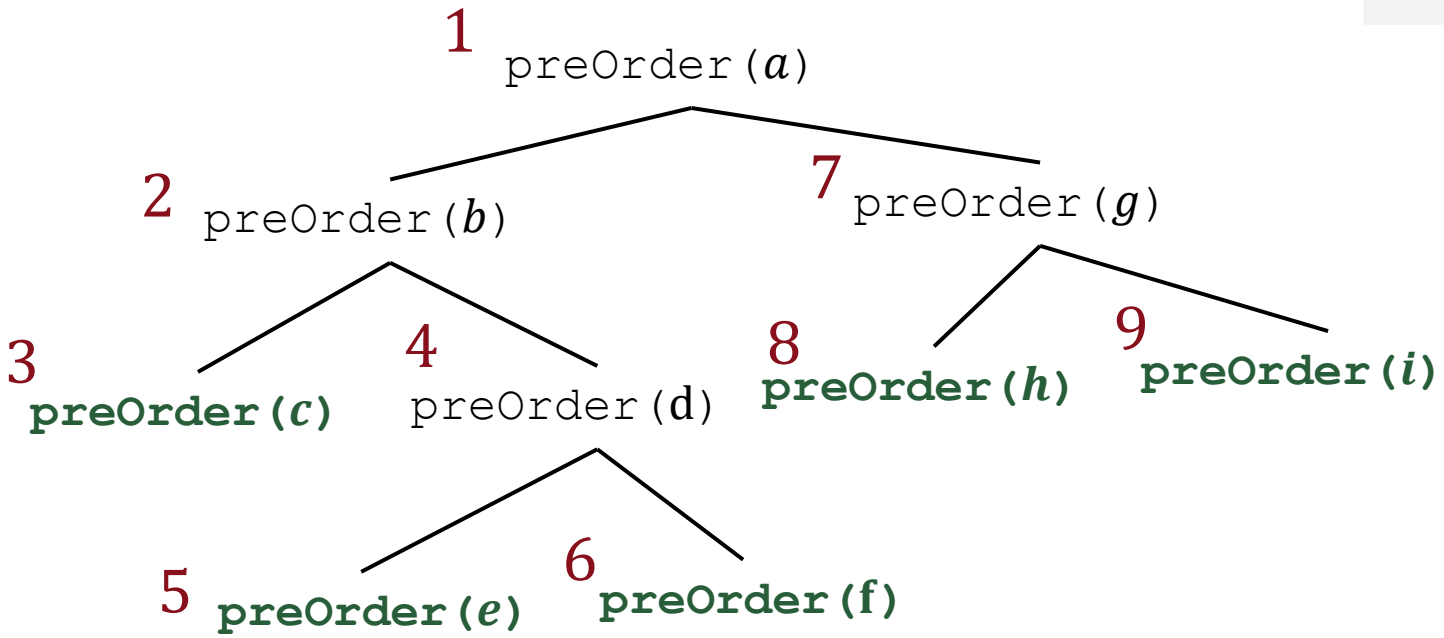
- Dummy node approach: add dummy nodes for missing children and skip them during visits
- Null check approach:

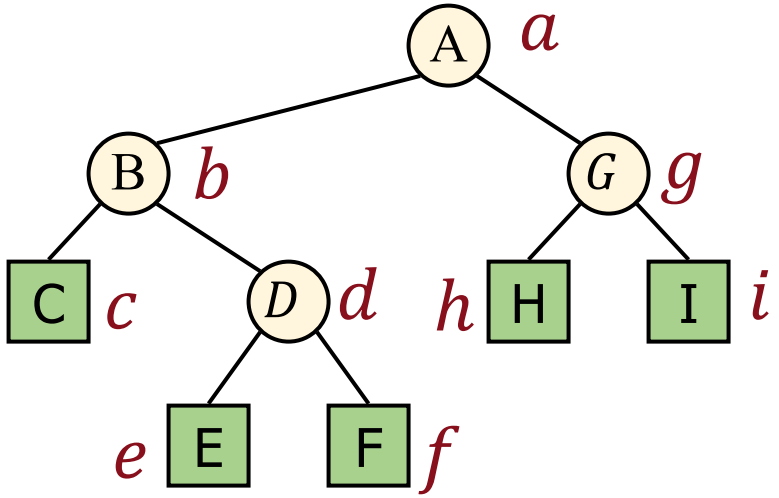
```
Algorithm preOrder(v)
  If v is null then
    return
  visit v
  preOrder(leftChild(v))
  preOrder(rightChild(v))
```

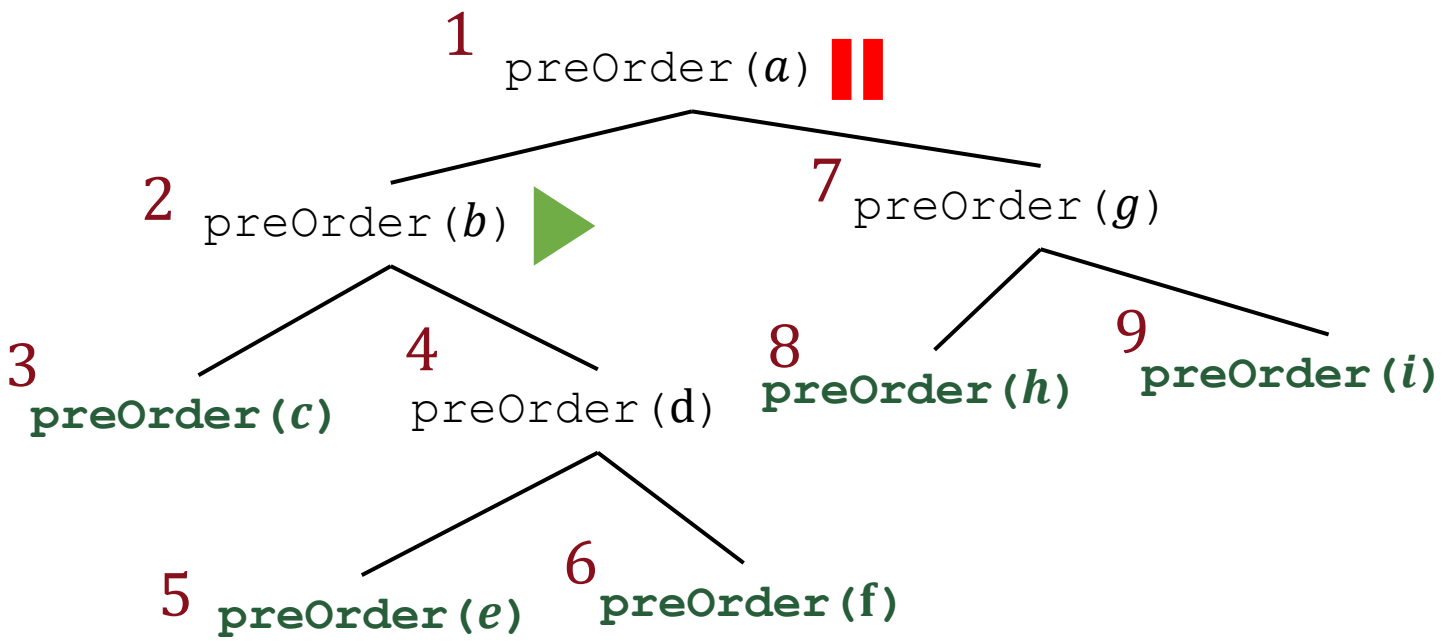
Preorder Traversal: Performance

- Time efficiency: $O(n)$
- Space efficiency: depending on space used by call stack in recursion

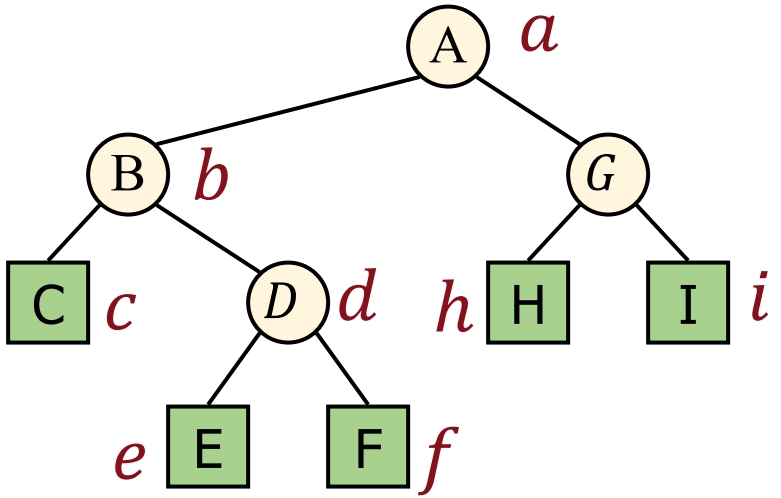
```
Algorithm preOrder(v)
    visit v
    If v is internal then
        preOrder(leftChild(v))
        preOrder(rightChild(v))
```

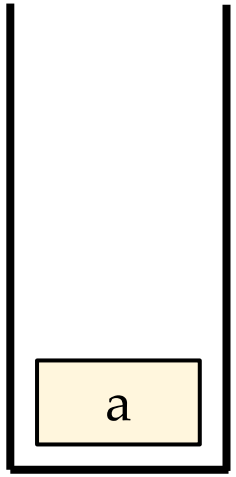




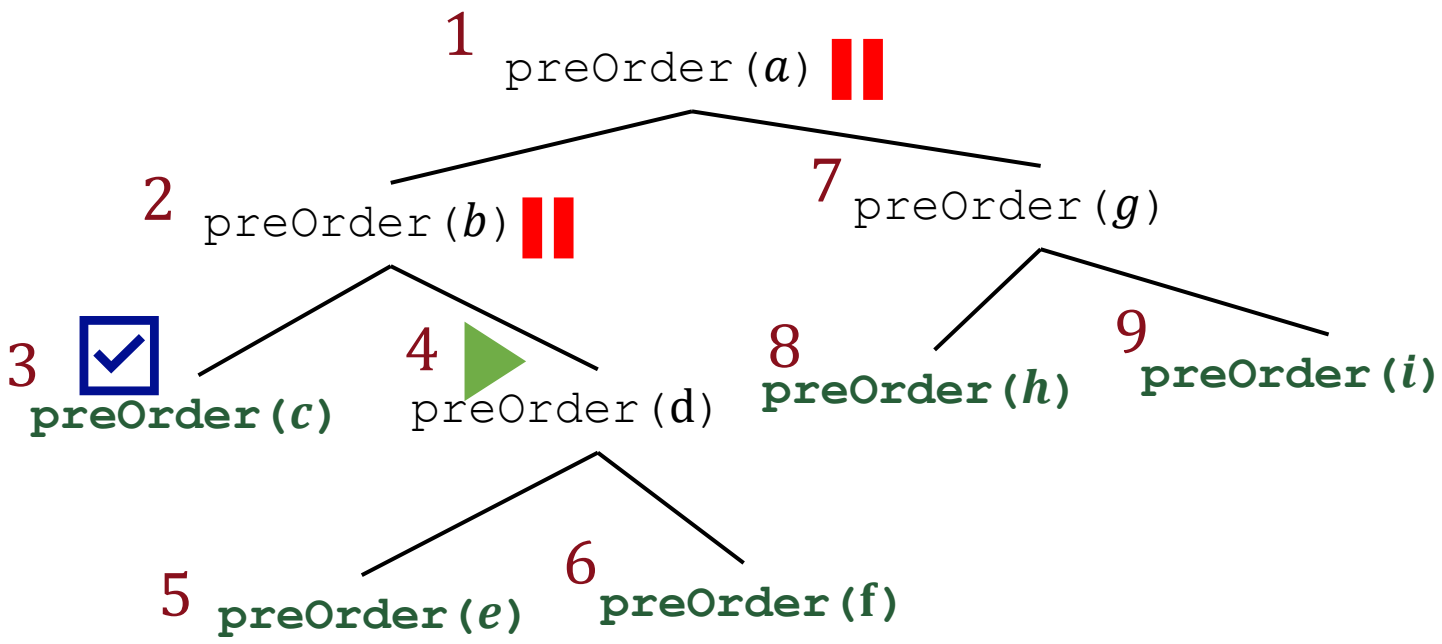



```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

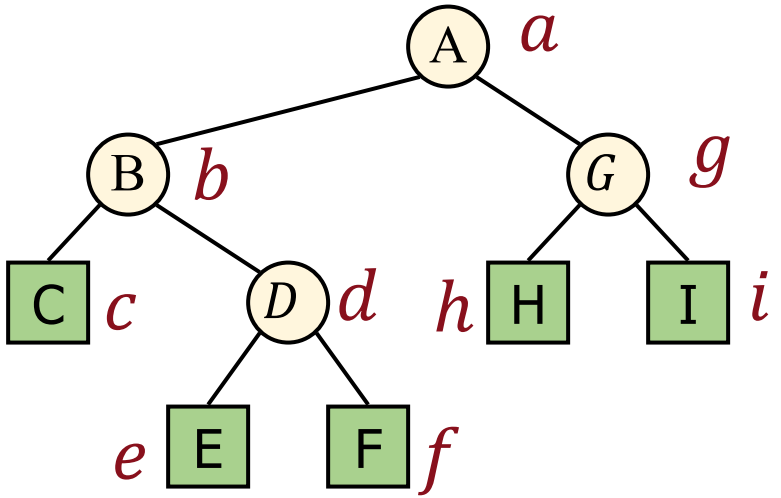


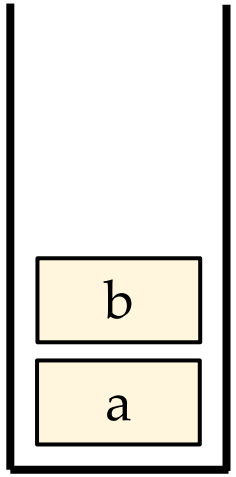


Call stack

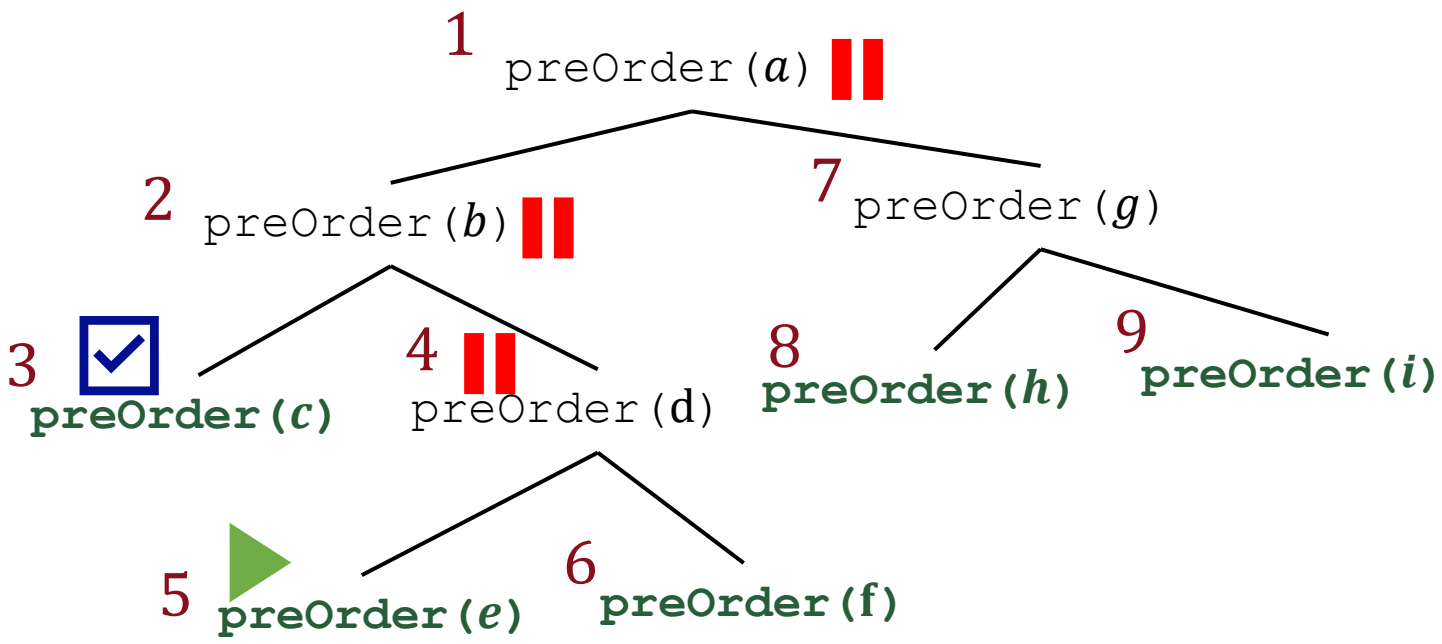



```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

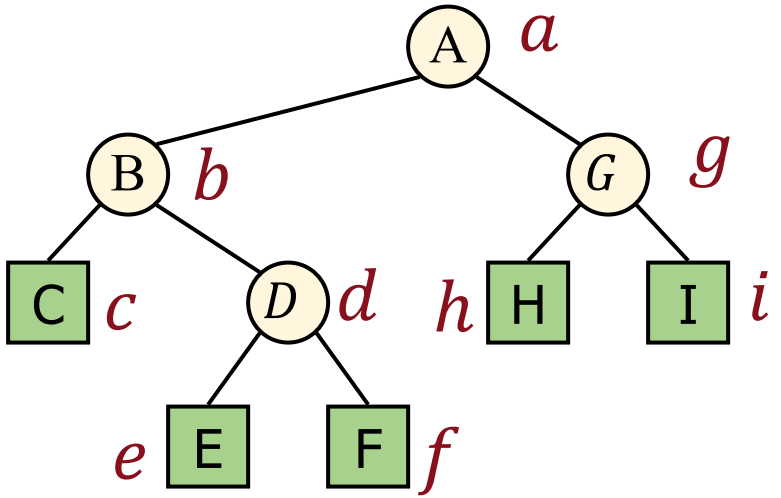


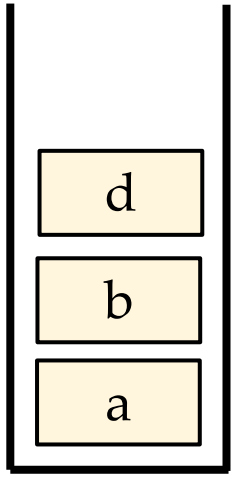


Call stack

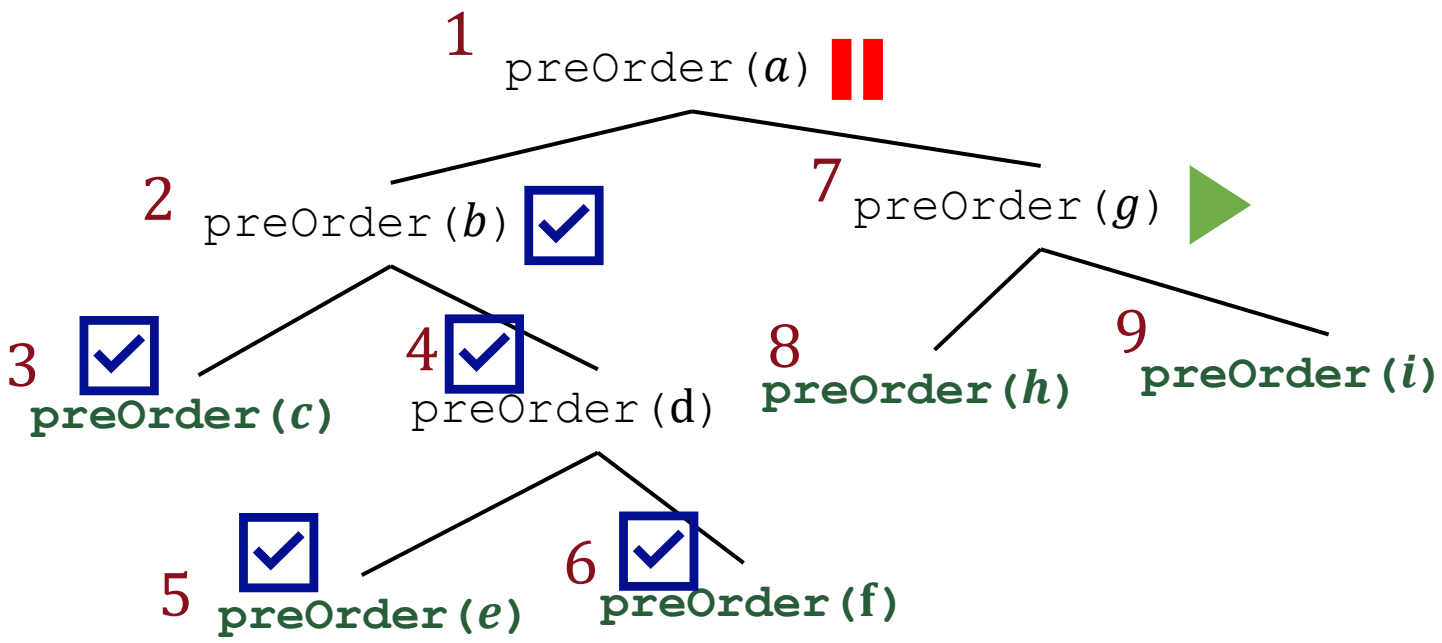



```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

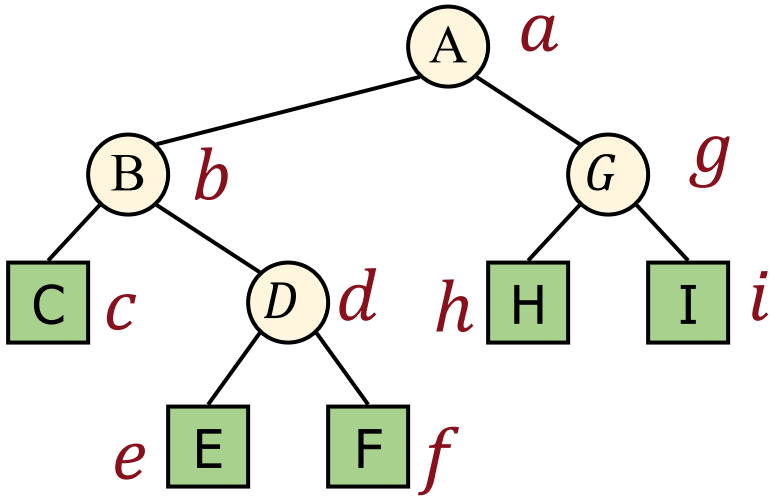


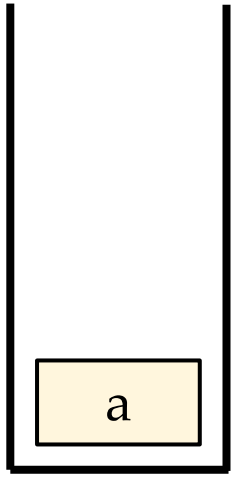


Call stack

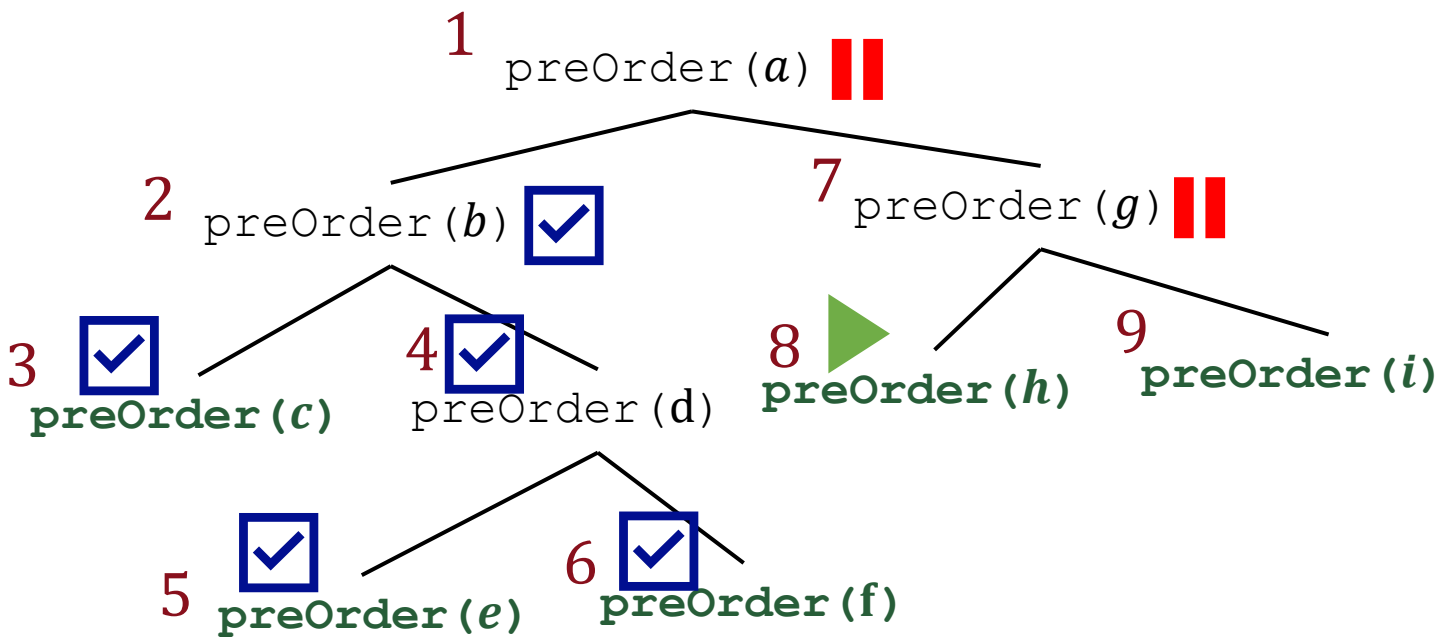



```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

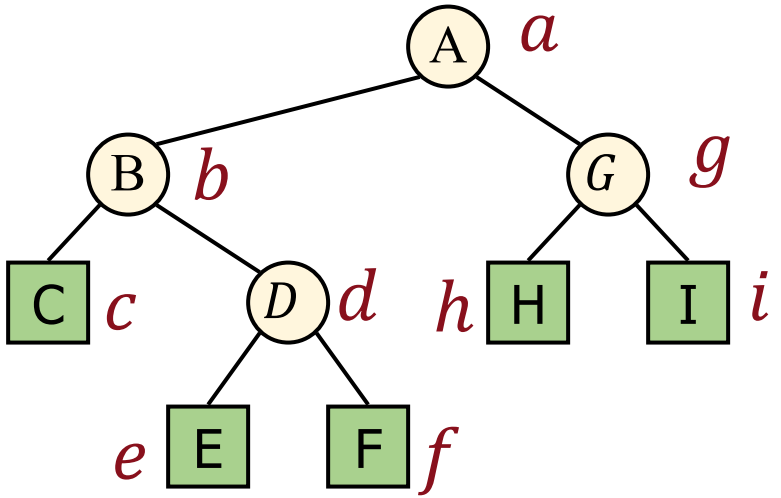


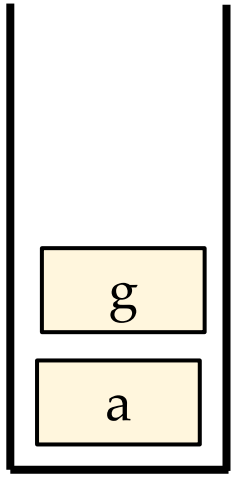


Call stack

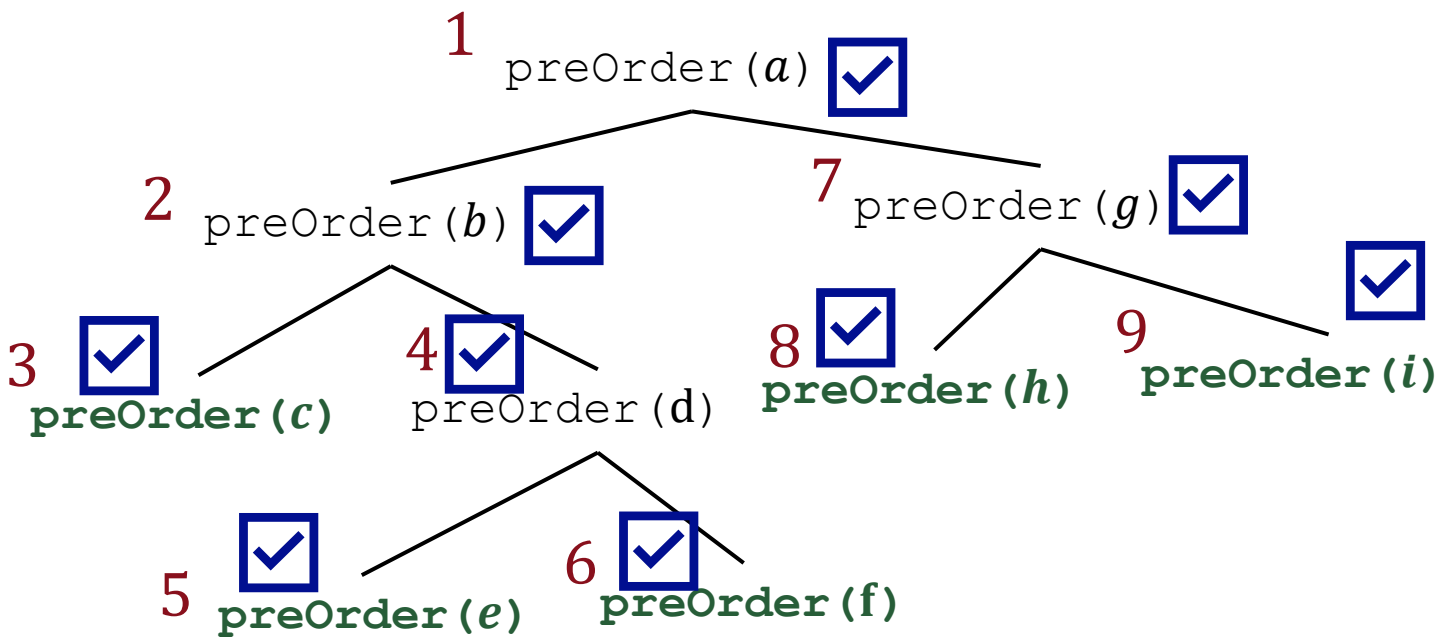


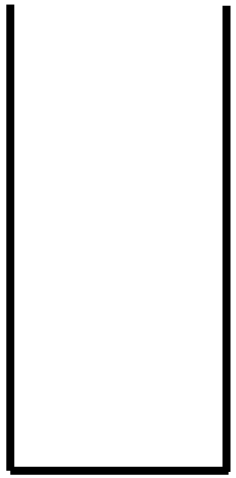

```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```





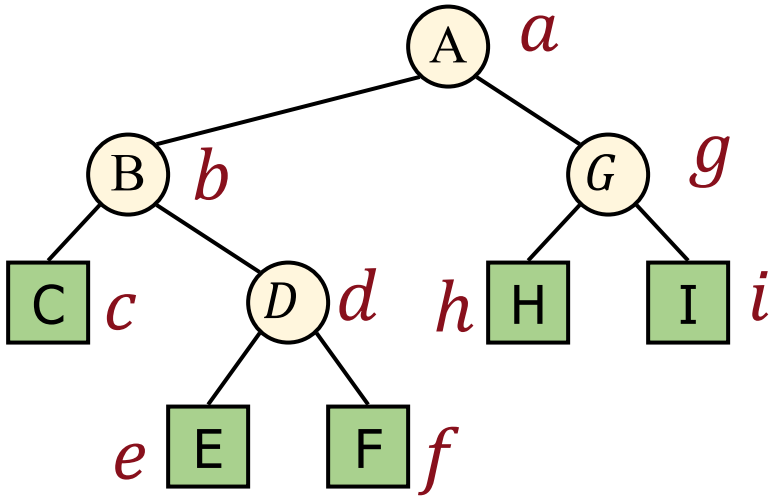
Call stack





Call stack

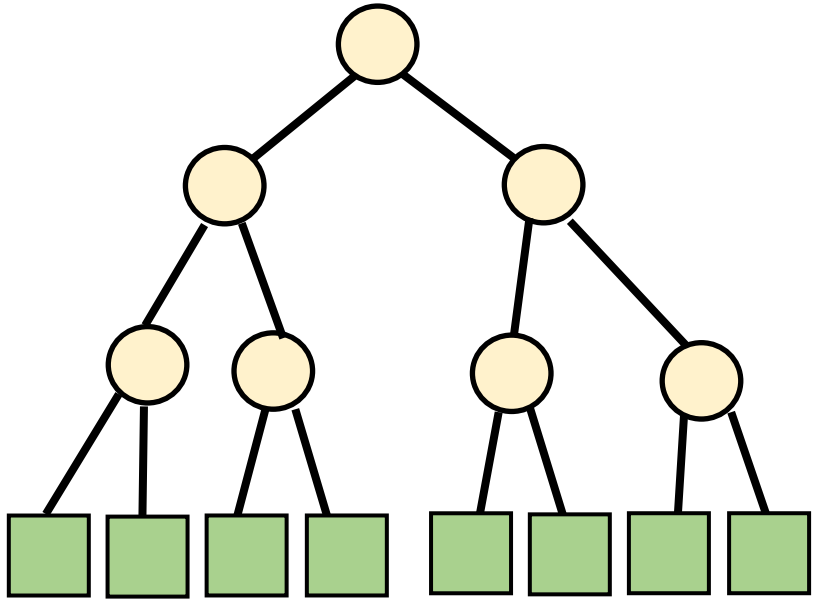
```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```



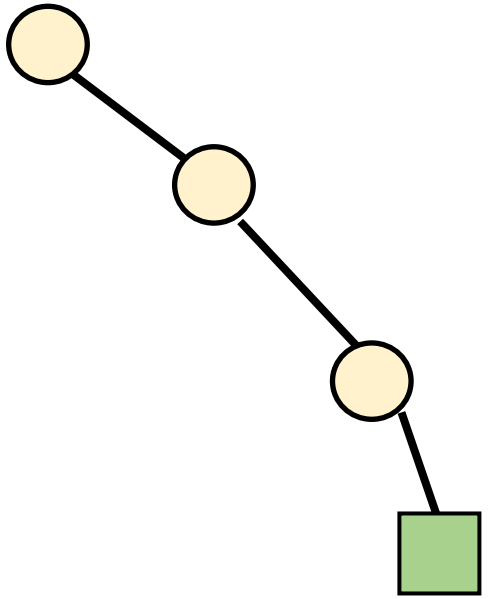
Preorder Traversal: Performance

- Time efficiency: $O(n)$
- Space efficiency: the extra space used is proportional to the height of the tree

```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```



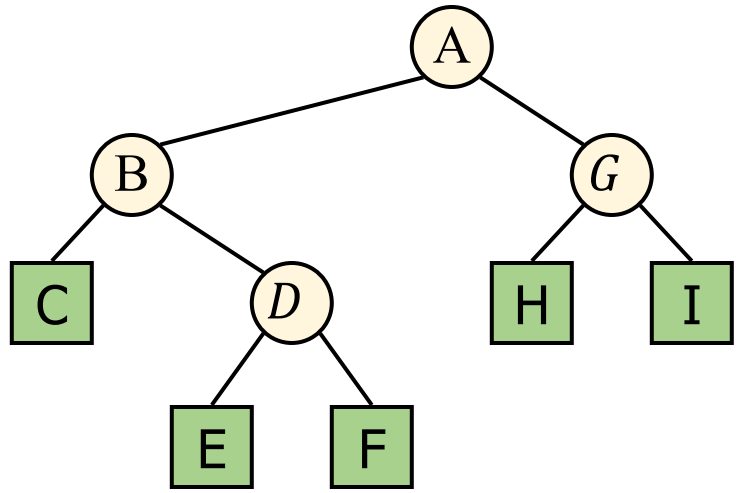
Best case: $O(\log n)$



Worst case: $O(n)$

Application of tree traversals: height of a subtree

- Given a position v, return its height of the subtree rooted at v
- Hint: If we know the heights of the left and right subtrees, the height of the current subtree is

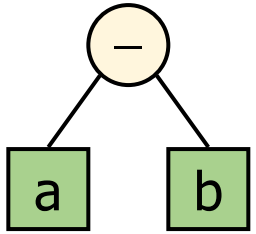


$$\text{height}(v) = 1 + \max\{\text{height}(\text{leftChild}(v)), \text{height}(\text{rightChild}(v))\}$$

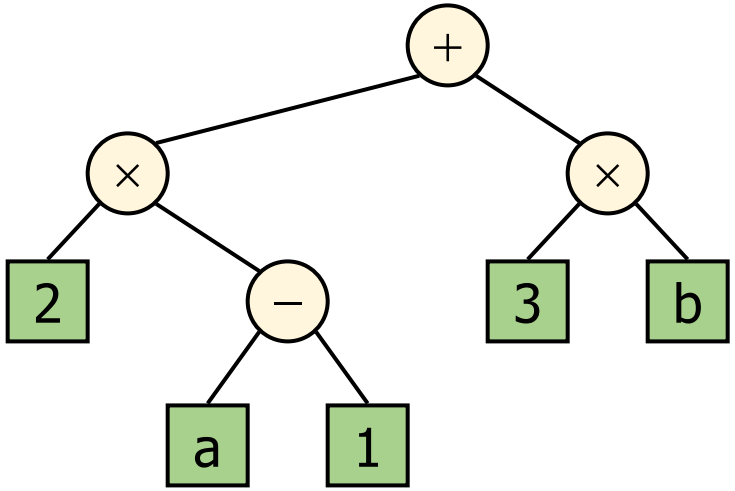
- Customize postorder traversal:

```
Algorithm height(v)
  If isInternal(v) then
    return 1 + max{height(leftChild(v)), height(rightChild(v))}
  Else
    return 0
```

Application of tree traversals: arithmetic expression



Inorder: a – b
Postorder: a b –
Preorder – a b

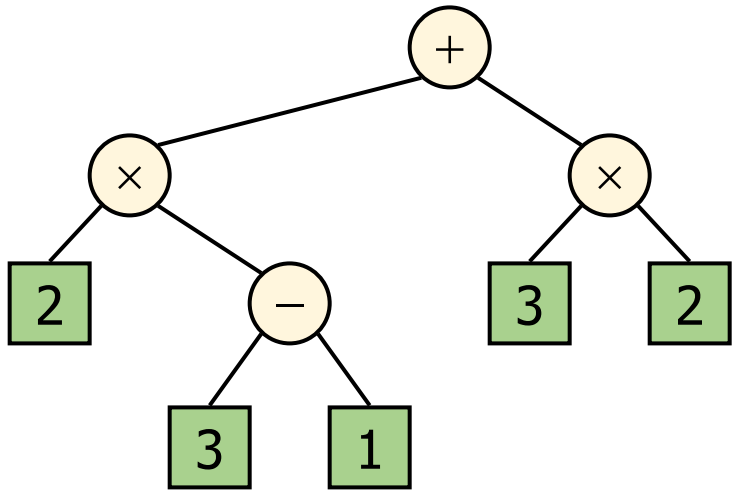


Inorder:
2 × a - 1 + 3 × b

Postorder:
2 a 1 - × 3 b × +

Print arithmetic expression

- Customize inorder traversal:
  - print operand or operator when visiting a node
  - print "(" before traversing left subtree
  - print ")" after traversing right subtree



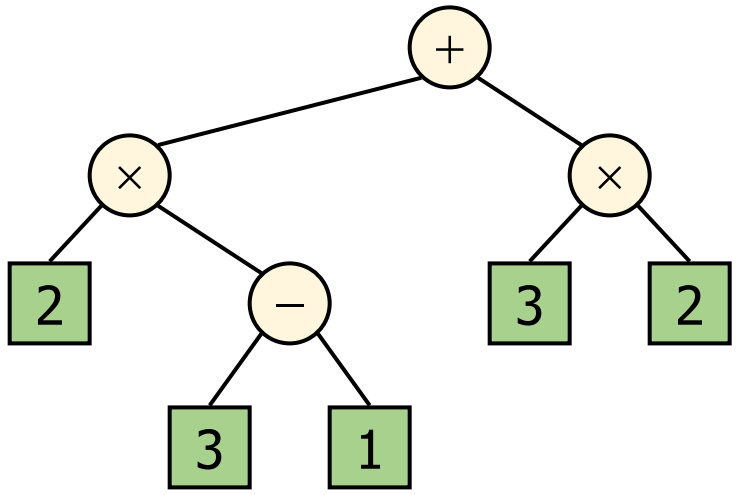
$2 \times 3 - 1 + 3 \times 2$

$((2 \times (3 - 1)) + (3 \times 2))$

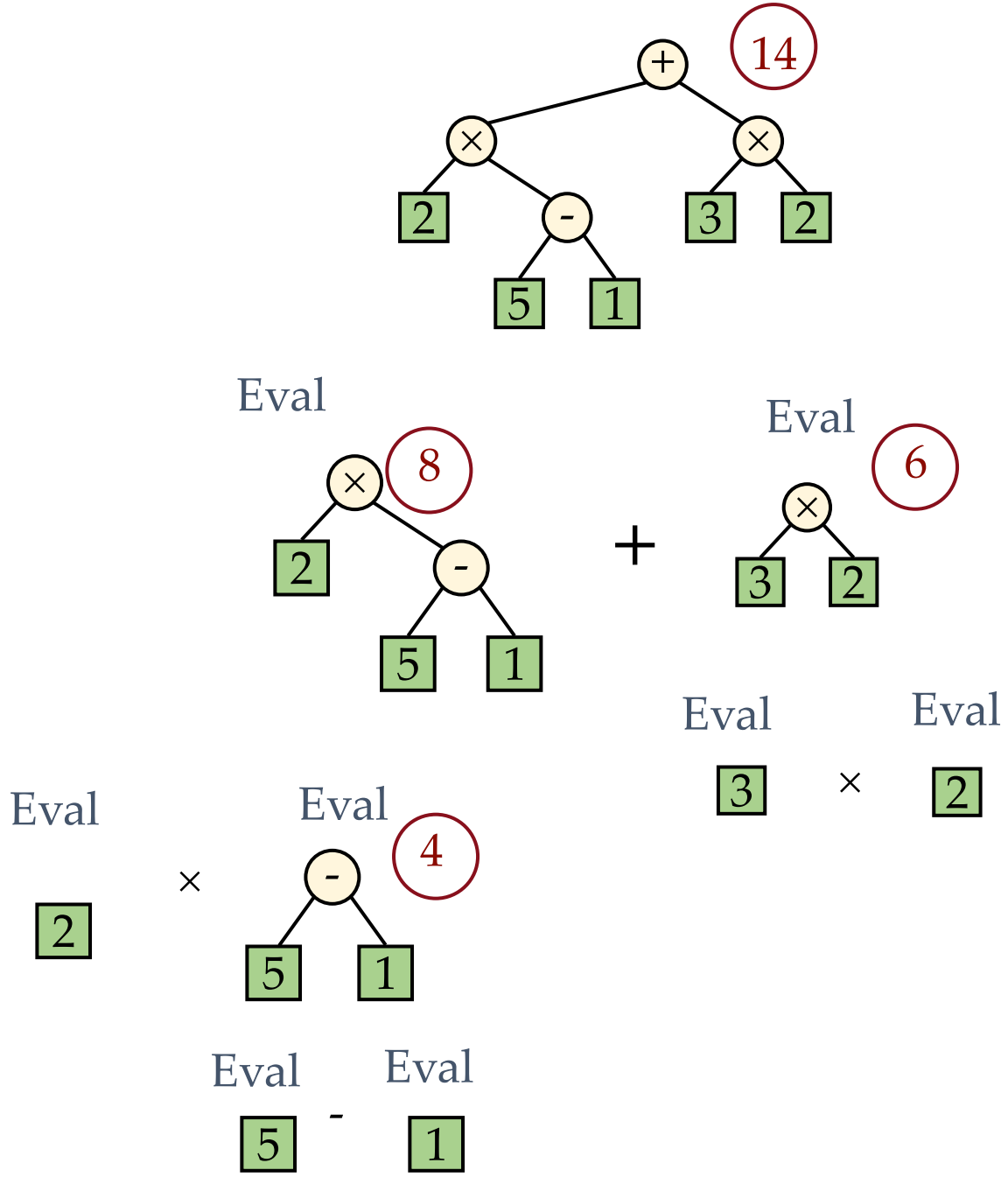
```
Algorithm printExpr(v)
  If isInternal(v) then
     print("(")
     printExpr(leftChild(v))
  print(v.element)
  If isInternal(v) then
     printExpr(rightChild(v))
     print(")")
```

Evaluate arithmetic expression

- Customize postorder traversal:
  - recursive method returning the value of a subtree
  - when visiting an internal node, combine the values of the subtrees




```
Algorithm evalExpr(v)
    If isExternal(v) then
        return v.element
    Else
        x ← evalExpr(leftChild(v))
        y ← evalExpr(rightChild(v))
        ◇ ← v.element
        return x ◇ y
```



Algorithm `evalExpr(v)`

If `isExternal(`v`)` **then**

`return` v.element

Else

$x \leftarrow$ `evalExpr(`leftChild(v)`)`

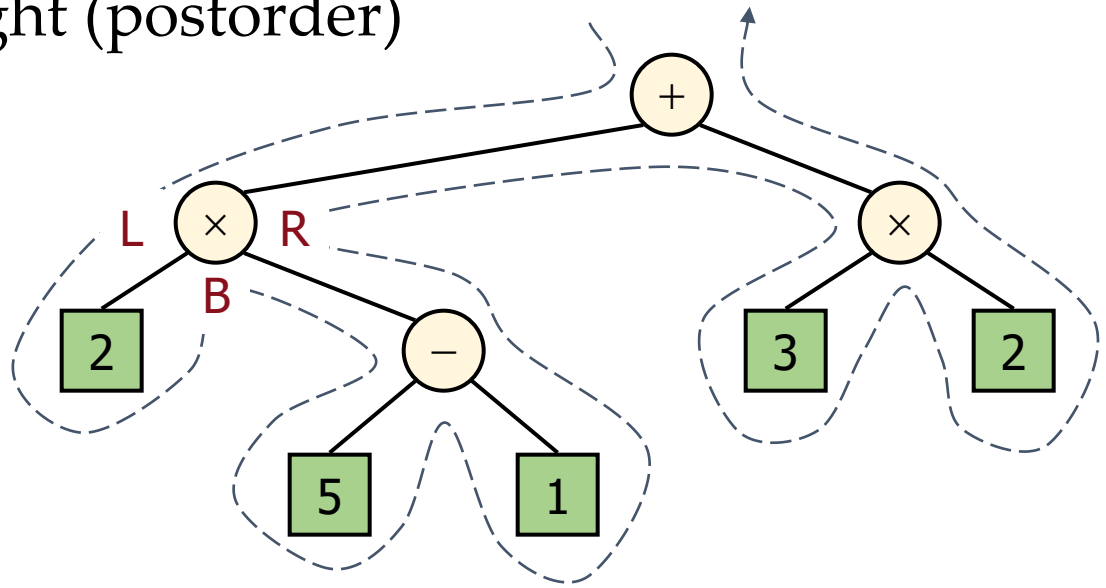
$y \leftarrow$ `evalExpr(`rightChild(v)`)`

$\diamond \leftarrow$ v.element

`return` $x \diamond y$

Euler tour traversal

- Generic traversal of a binary tree
- Includes the preorder, postorder and inorder traversals as special cases
- Walk around the tree and visit each node three times:
  - on the left (preorder)
  - from below (inorder)
  - on the right (postorder)



Euler tour traversal

```
Algorithm preOrder(v)
  visit v
   If v is internal then
      preOrder(leftChild(v))
      preOrder(rightChild(v))
```

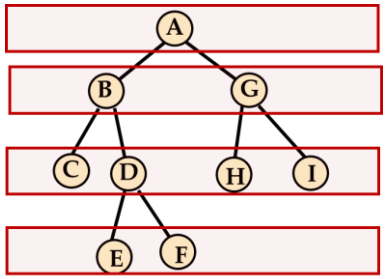
```
Algorithm postOrder(v)
   If v is internal then
      postOrder(leftChild(v))
      postOrder(rightChild(v))
   visit v
```

```
Algorithm inOrder(v)
  If v is internal then
     inOrder(leftChild(v))
  visit v
  If v is internal then
     inOrder(rightChild(v))
```

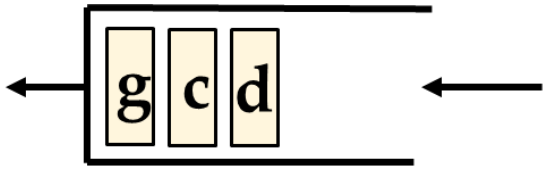
```
Algorithm eulerTour(v)
  leftVisit v
  If v is internal then
     eulerTour(leftChild(v))
  belowVisit v
  If v is internal then
     eulerTour(rightChild(v))
  rightVisit v
```

Summary
Definition, Algorithm, and Performance

Breadth-first traversal (level by level)



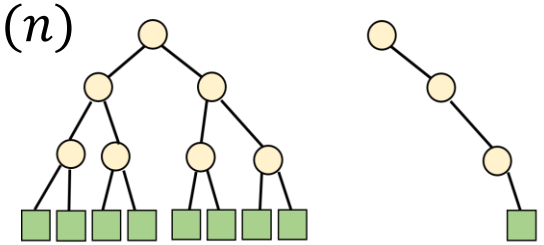
Queue-based algorithm



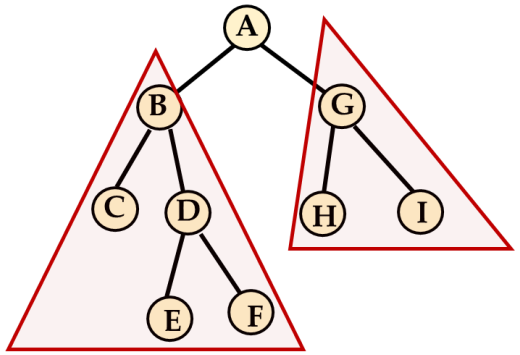
Dequeue a node then enqueue its children

Time efficiency $O(n)$

Space efficiency



Depth-first traversal (recursively visit root and subtrees)

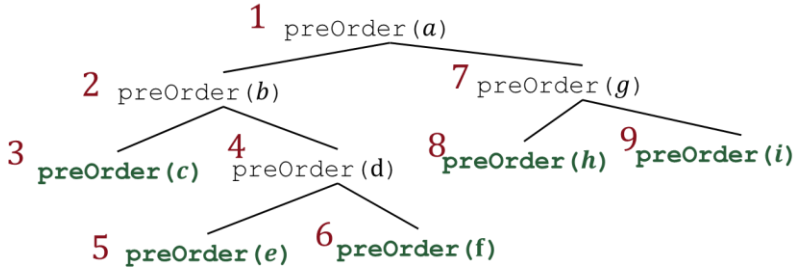


Preorder: root → left → right

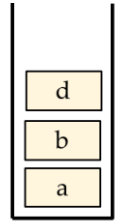
Postorder: left → right → root

Inorder: left → root → right

Recursive algorithm



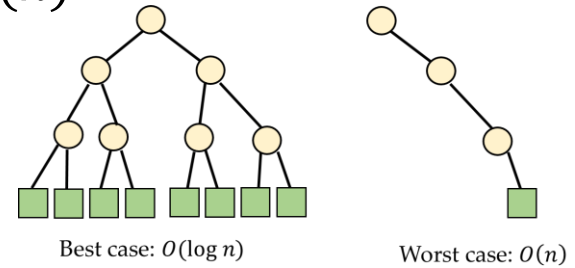
Trace of recursive calls



Call stack

Time efficiency $O(n)$

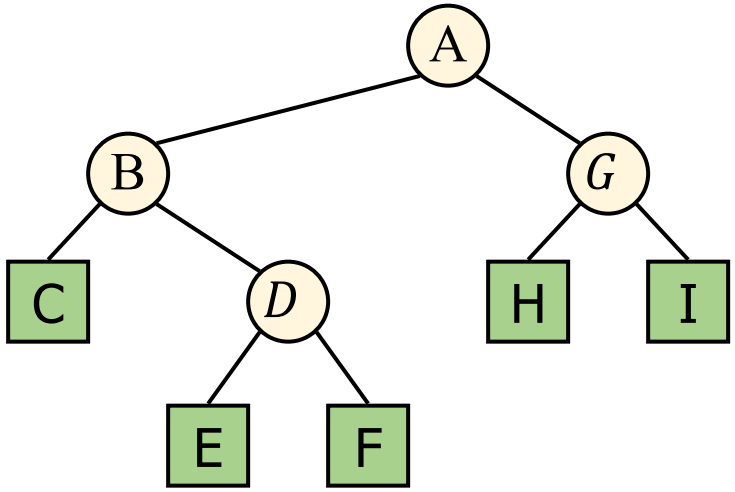
Space efficiency



Exercise: preorder traversal with stack

```
Algorithm preOrderwithStack(v)
 S ← new stack
 S.push(v)

While S is not empty do
    N = S.pop()
    If N != null then
      visit N
      S.push(N.right)
      S.push(N.left)
```

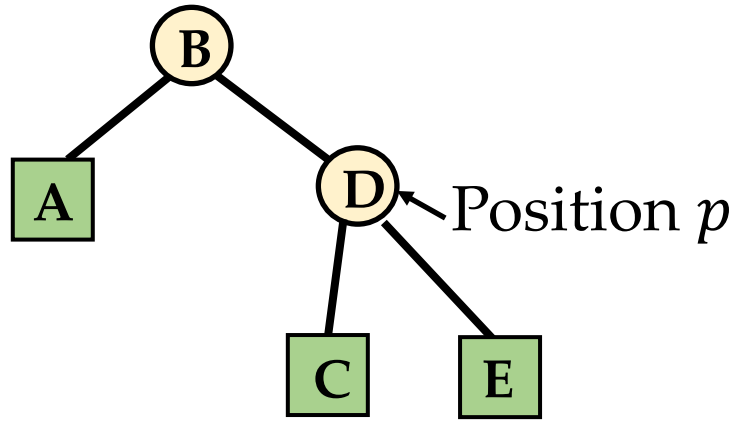


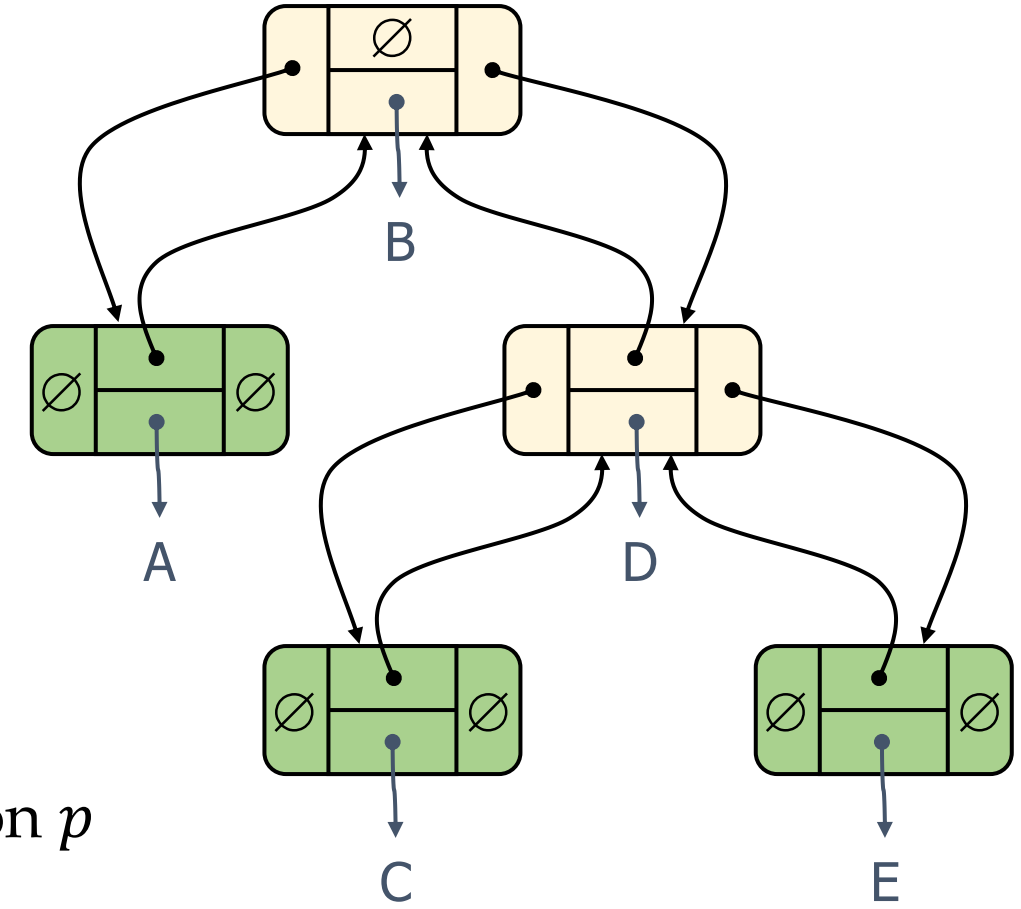
Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - Binary Tree property
- Traversals of Trees
  - Tree traversals
  - Binary tree traversals
- Implementing Tree ADT
  - Binary tree
  - General tree

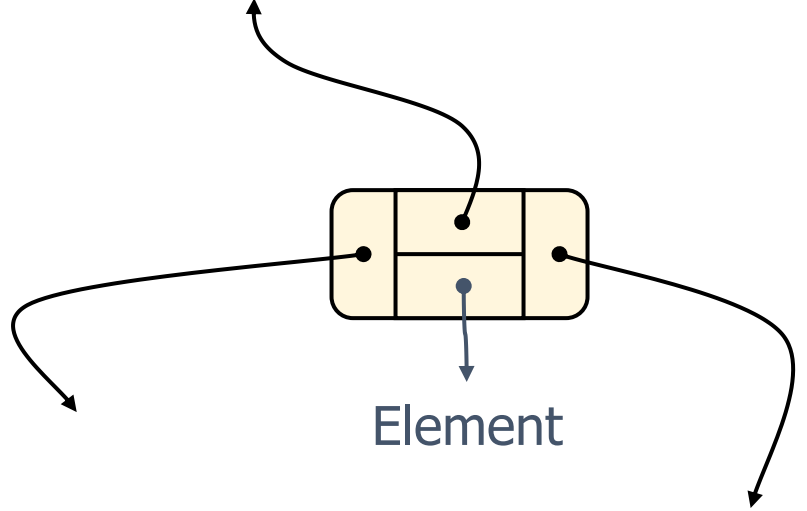
Implementing binary trees with linked structure

- Nodes implement positions and store
  - Element
  - Link to parent node
  - Link to left child node
  - Link to right child node





Implemented Binary Trees



Algorithm `leftChild(v)`

```
return v.left
```

Algorithm `rightChild(v)`

```
return v.right
```

Algorithm `sibling(v)`

```
p ← v.parent
q ← p.left
If v = q then
  return p.right
Else
  return q
```

Algorithm `swapElement(v,w)`

```
temp ← v.element
v.element ← w.element
w.element ← temp
```

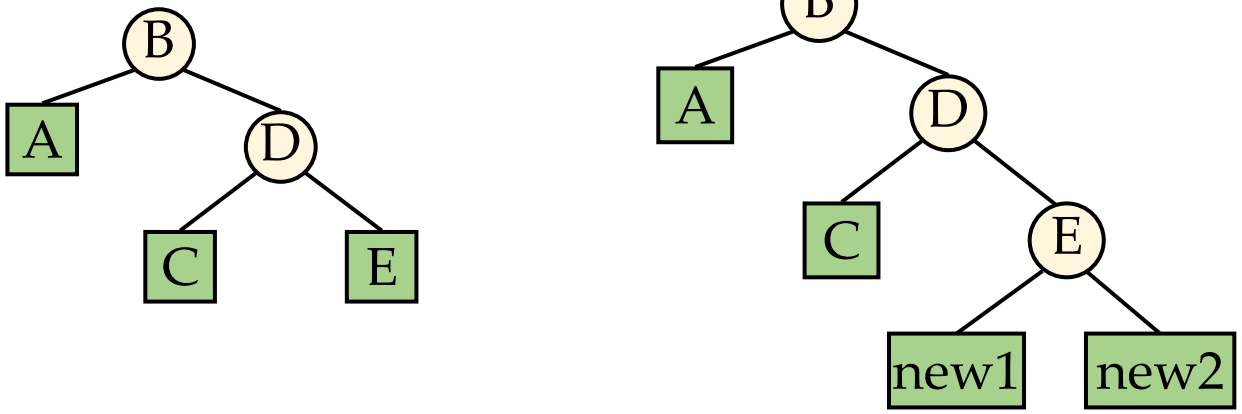
`parent(p)`, `replaceElement(p,e)`, `isRoot(p)`, `isExternal(p)`, `isExternal(p)`

Time complexity: $O(1)$

Insertion: expandExternal(v)

- expandExternal(v): Transform v from an external node into an internal node by creating two new children

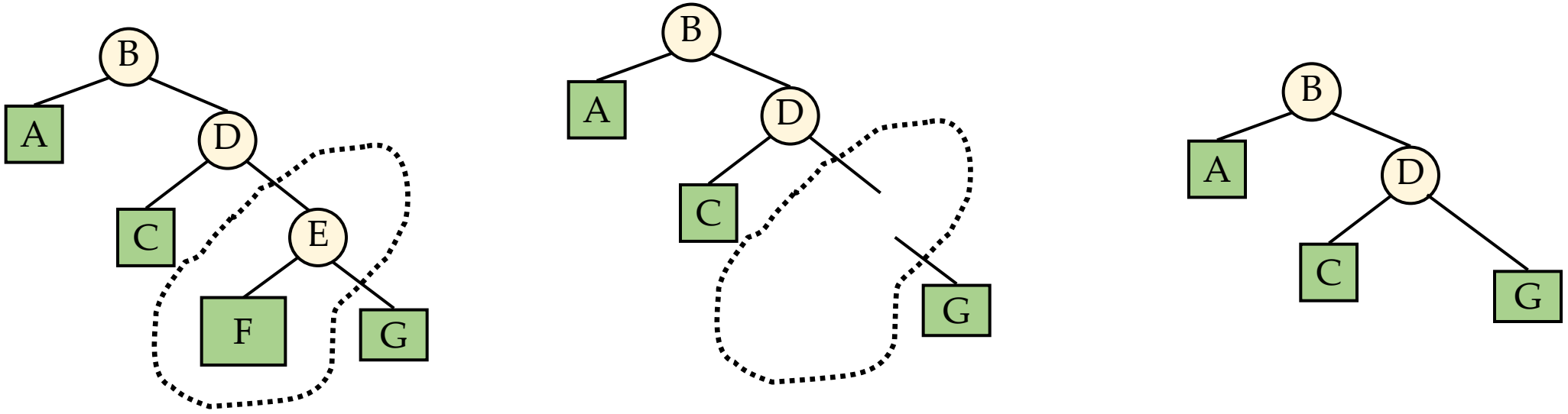
```
Algorithm expandExternal(v)
    If isExternal(v) then
        create new nodes new1 and new2
        v.left ← new1
        v.right ← new2
        size ← size + 2
```



Time complexity: $O(1)$

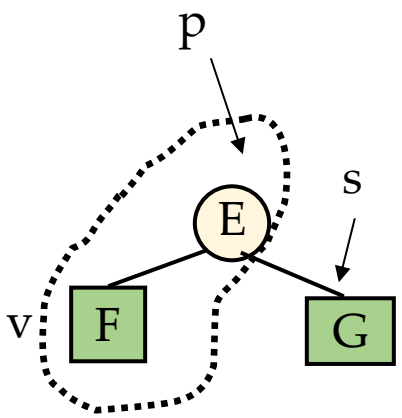
Deletion: removeExternal(v)

- removeExternal(v): remove the external node at position v and its parent, replacing its parent with node v's sibling

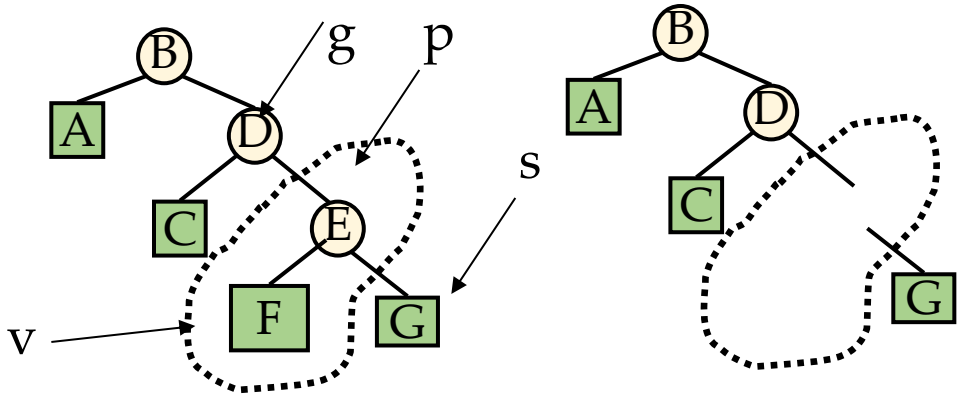


Deletion: removeExternal(v)

Parent is root



Parent is not root



Algorithm removeExternal(v)

If isExternal(v) and size ≥ 3 **then**

$p \leftarrow$ parent(v)

$s \leftarrow$ sibling(v)

If isRoot(p) **then**

s.parent $\leftarrow$ null

root $\leftarrow s$

Else

$g \leftarrow$ parent(p)

If p = leftChild(g) **then**

g.left $\leftarrow s$

Else

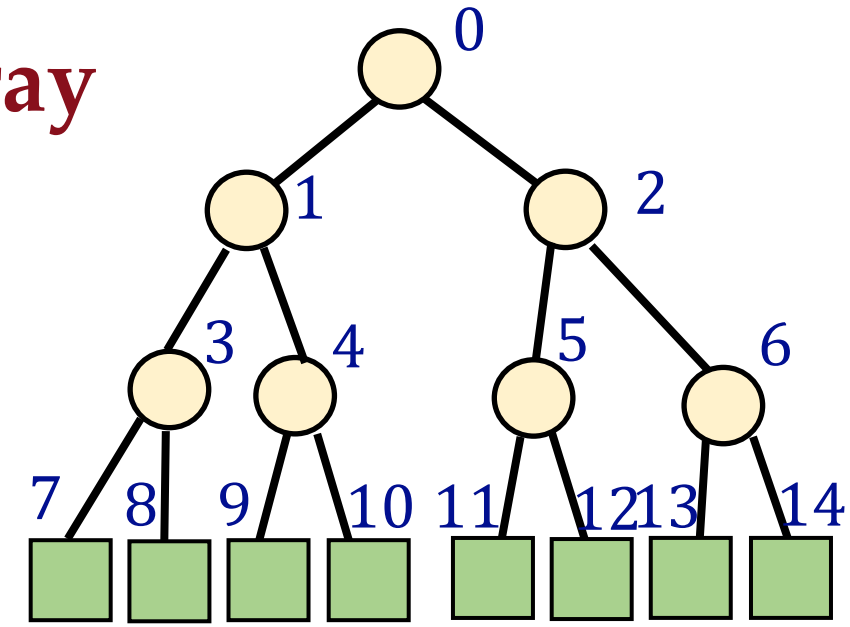
g.right $\leftarrow s$

$size \leftarrow size - 2$

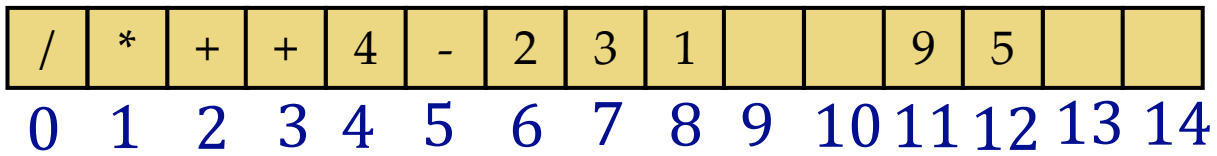
Time complexity: $O(1)$

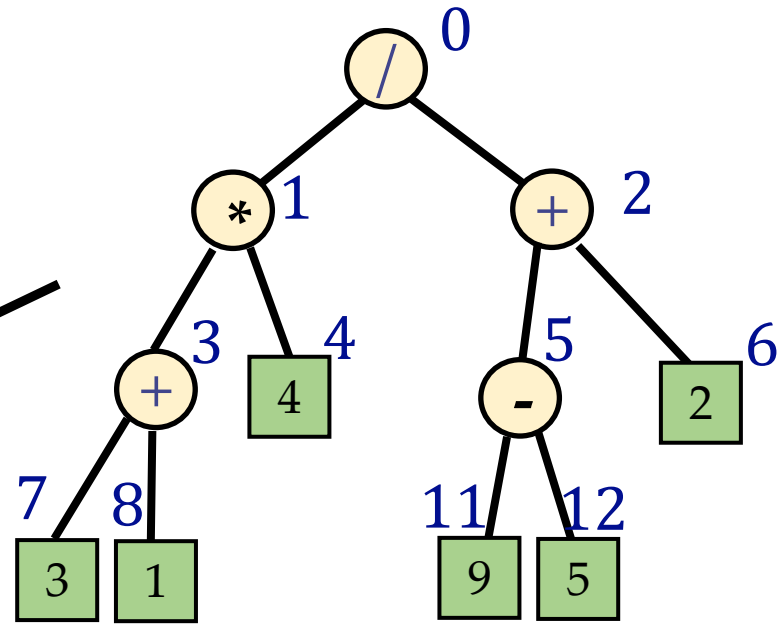
Implementing binary trees using array

- Number each position p using a numbering function $f(p)$
  - If p is the root, then $f(p) = 0$
  - If p is the left child of q, then $f(p) = 2f(q) + 1$
  - If p is the right child of q, then $f(p) = 2f(q) + 2$
- Store the element at position p at index $f(p)$ of an array



Numbering can start from 1 instead of 0 and the numbering function is different





Array-based binary tree

Algorithm leftChild(i)

  If $2i+1 \leq n$ **then**

    return $2i+1$

Algorithm rightChild(i)

  If $2i+2 \leq n$ **then**

    return $2i+2$

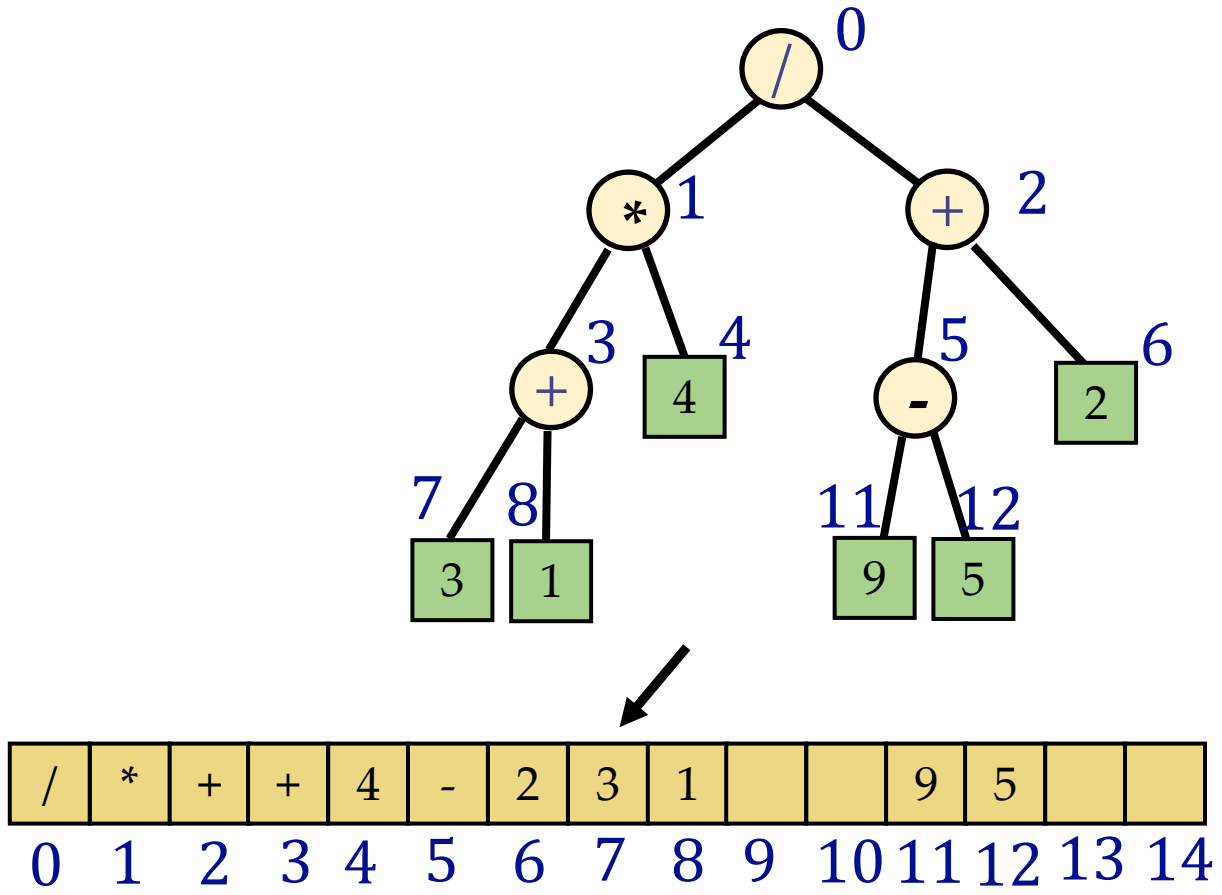
Algorithm parent(i)

  If $i \geq 1$ **then**

    return $\lfloor (i-1)/2 \rfloor$

  Else

    return null

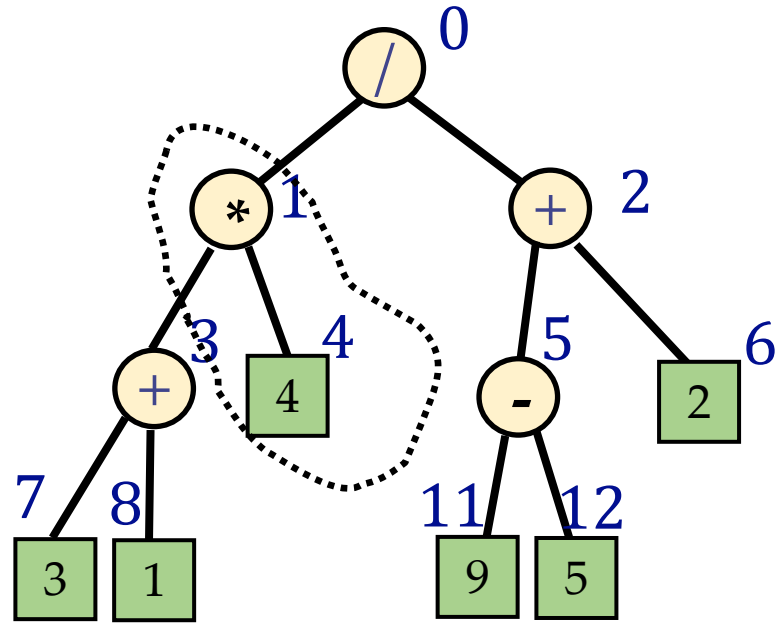


sibling(i), replaceElement(i, e),
isRoot(i), isExternal(i), isExternal(i)

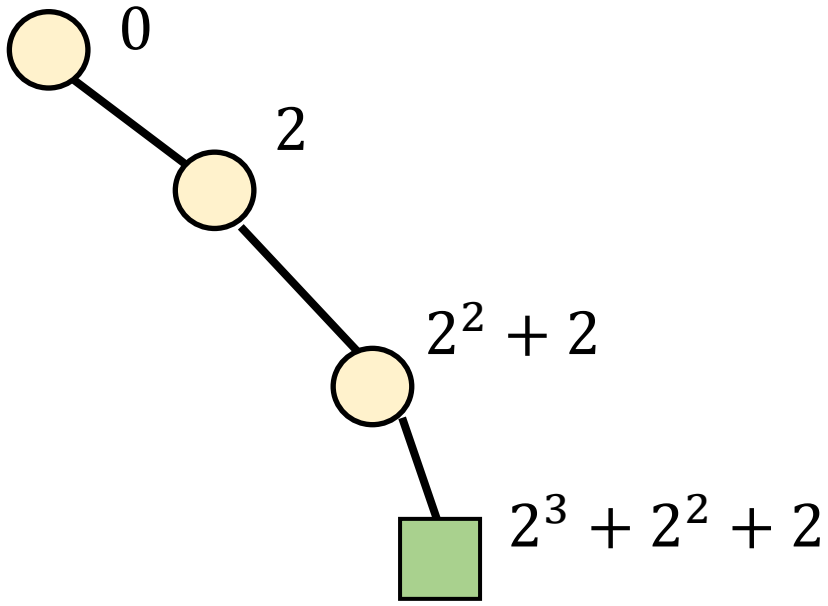
Time complexity: $O(1)$

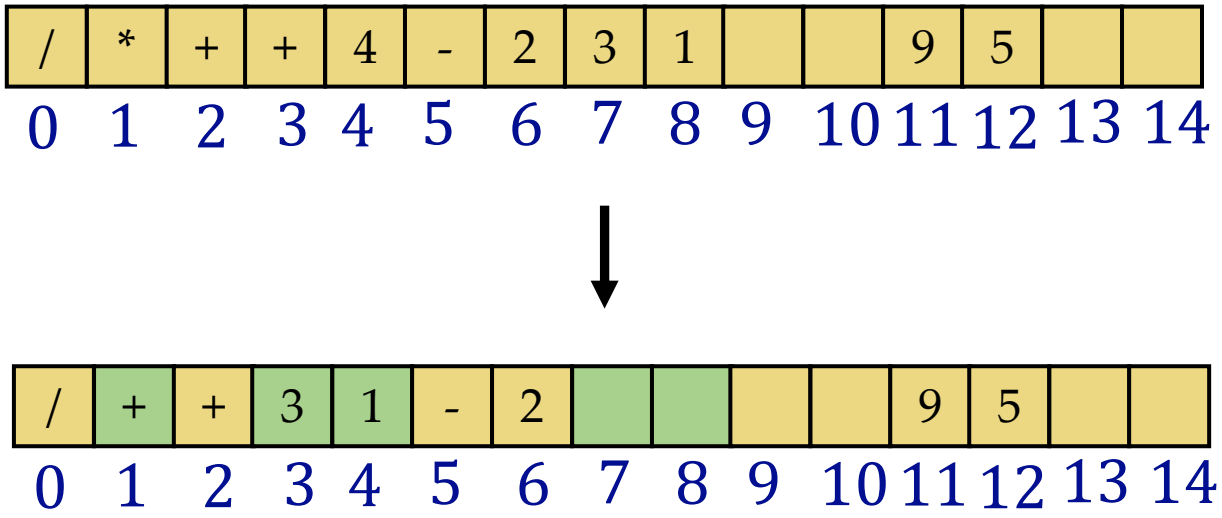
Array-based binary tree: drawbacks

- removeExternal(i) is more difficult to implement because it is not just the child that moves, but all descendants of that child



- In the worst case, a tree with n needs an array of length $2^n - 1$





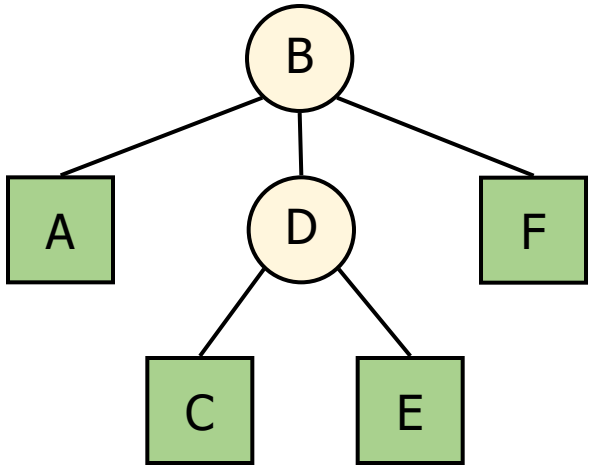
index of the last node $2^{n-1} + 2^{n-2} + \cdots + 2 = 2^n - 2$

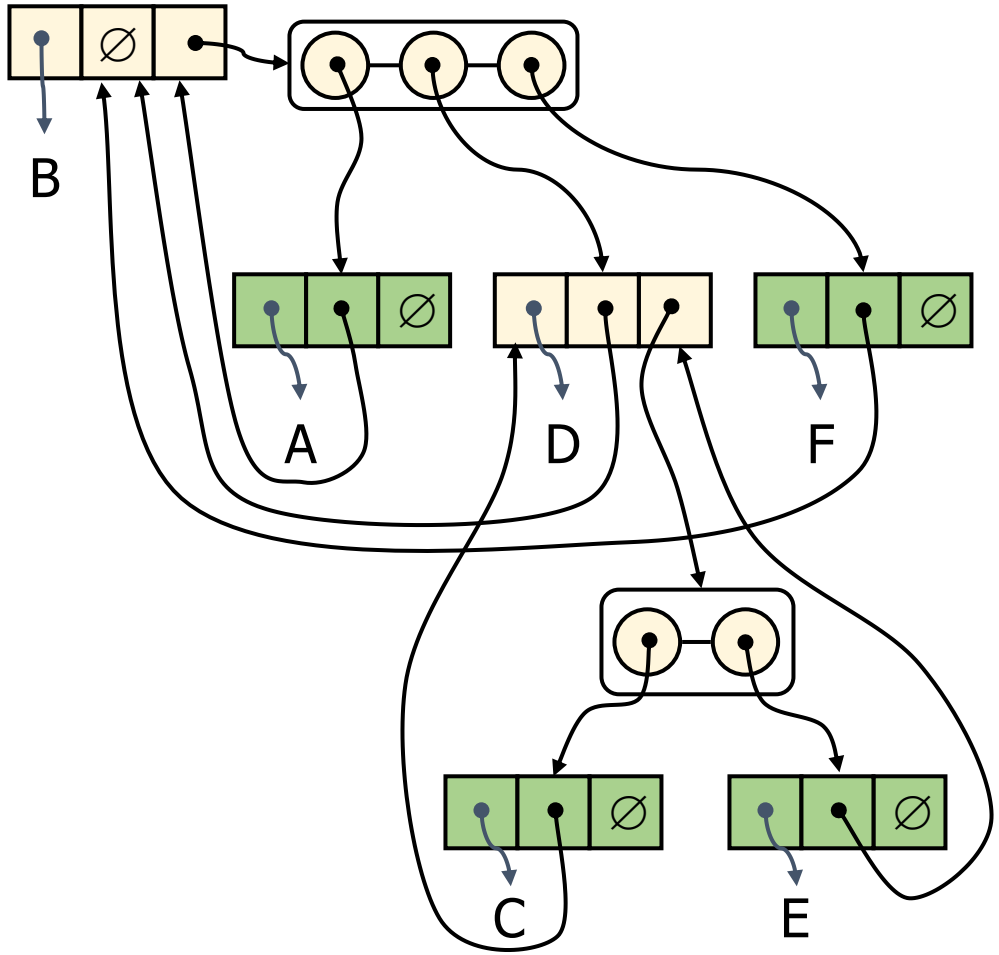
Trees and Binary Trees: roadmap

- Trees and Binary Trees
  - Tree
  - Binary trees
- Properties of Trees
  - Tree property
  - Binary Tree property
- Traversals of Trees
  - Tree traversals
  - Binary tree traversals
- Implementing Tree ADT
  - Binary tree
  - General tree

Implementing general trees with linked structure

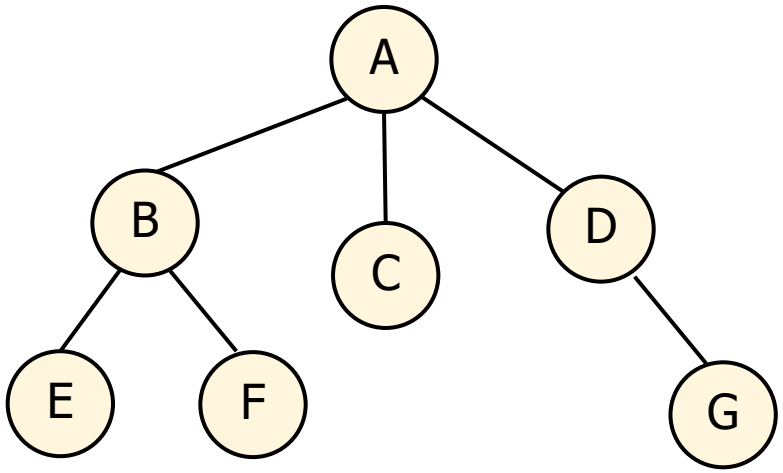
- Nodes implement positions and store
  - Element
  - Link to parent node
  - Link to children nodes

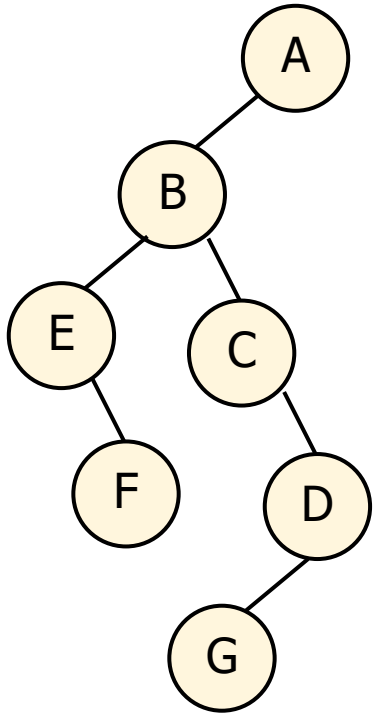




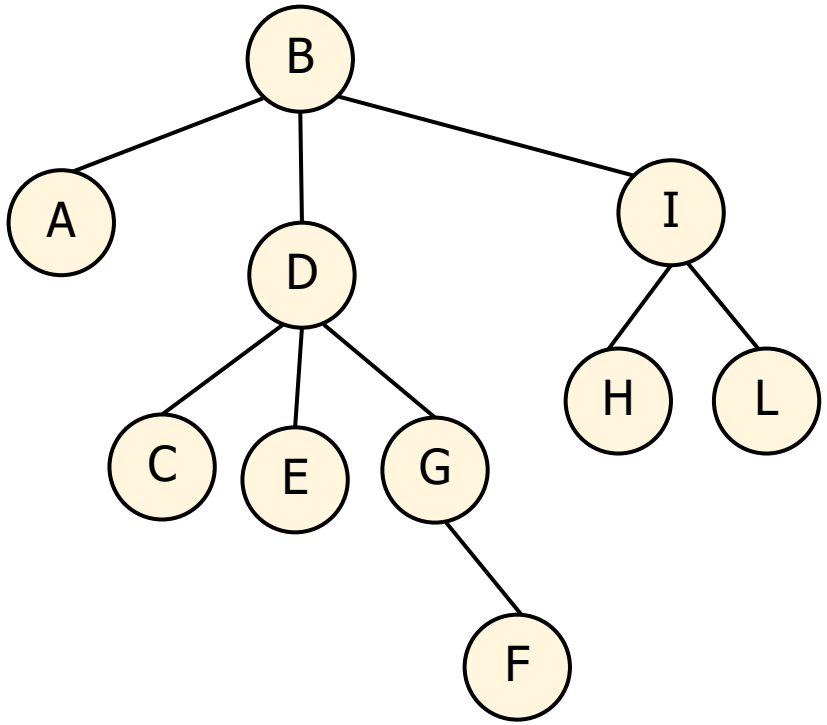
Representing general trees using binary trees

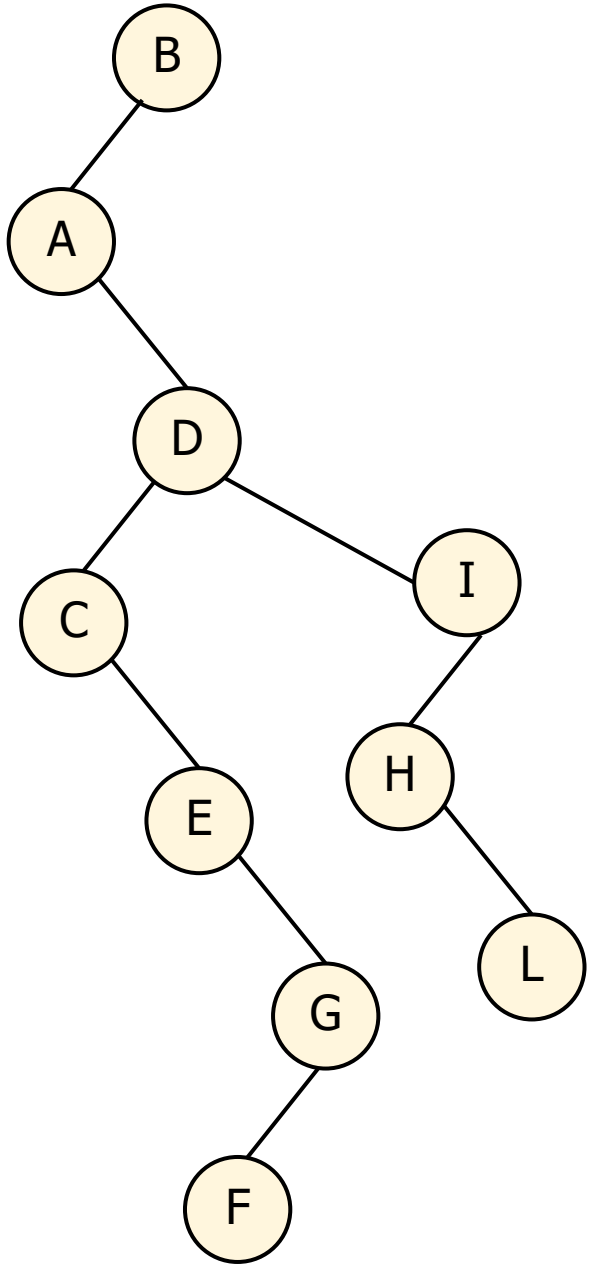
- Rules (node n in general tree and node n' in binary tree)
  - Root of binary tree is the root of general tree
  - Left child of n' is the first child of n
  - Right child of n' is the first sibling of n





Exercise





The same pre-order traversal

Next Topic

priority queue